

Criptografia

Módulo de Cibersegurança — MEEC



João Pavão

Universidade de Trás-os-Montes e Alto Douro

Criptografia

Módulo de Cibersegurança — Mestrado em Engenharia Eletrotécnica e de Computadores

João Pavão

2026-09-14

Índice

Prefácio	1
Organização da sebenta	1
Autor	1
1 Criptografia Clássica — Substituição, Polialfabética e Transposição	2
Introdução	2
1.1 Fundamentos dos Sistemas Criptográficos	2
1.1.1 Classificação dos sistemas criptográficos	2
1.1.2 O modelo de cifra simétrica	3
1.1.3 O princípio de Kerckhoffs	3
1.1.4 Força bruta e criptanálise	4
1.2 Cifras de Substituição	4
1.2.1 A cifra de César	5
1.2.2 Cifra monoalfabética	6
1.2.3 Cifra de Playfair	9
1.2.4 Cifra polialfabética: Vigenère	10
1.2.5 Cifra de Vernam	11
1.2.6 One-Time Pad	12
1.3 Cifras de Transposição	14
1.3.1 A técnica rail fence	14
1.3.2 Transposição colunar	14
1.4 Síntese do Capítulo	16
1.5 Para Pensar	16
Referências	16
2 Cifras de Bloco — Fundamentos, Feistel, DES e 3DES	18
Introdução	18
2.1 Fundamentos das Cifras de Bloco	18
2.1.1 O que é uma cifra de bloco	18
2.1.2 A cifra ideal, e porque é impraticável	19
2.1.3 Confusão e difusão	21
2.2 A Rede de Feistel	21
2.2.1 Exemplo com uma rede reduzida	22

2.3	DES — Data Encryption Standard	22
2.3.1	A força do DES	24
2.4	3DES — Triple DES	25
2.5	Síntese do Capítulo	25
2.6	Para Pensar	26
	Referências	26
3	AES — Estágios e Modos de Operação	28
	Introdução	28
3.1	Do DES ao AES: um Novo Paradigma	28
3.2	Estrutura Geral do AES	29
3.3	As Quatro Transformações de Ronda	29
3.3.1	SubBytes	30
3.3.2	ShiftRows	30
3.3.3	MixColumns	30
3.3.4	AddRoundKey	31
3.4	A Ronda Completa e a Expansão de Chave	32
3.5	Segurança do AES	32
3.6	Modos de Operação	33
3.6.1	ECB — Electronic Codebook	33
3.6.2	CBC — Cipher Block Chaining	33
3.6.3	CFB — Cipher Feedback Mode	34
3.6.4	CTR — Counter Mode	34
3.6.5	GCM — Galois/Counter Mode	35
3.6.6	Comparação dos modos de operação	35
3.7	Síntese do Capítulo	36
3.8	Para Pensar	36
	Referências	36
4	Cifras de Fluxo — RC4, Salsa20, ChaCha20 e Padding	38
	Introdução	38
4.1	Cifras de Fluxo: o Conceito Geral	38
4.2	RC4	39
4.3	Duas Arquitecturas: Stateful vs. por Contador	40
4.4	Salsa20	41
4.5	ChaCha20	43
4.6	Padding	44
4.7	Síntese do Capítulo	44
4.8	Para Pensar	44
	Referências	45
5	Hashing — Propriedades, Ataques, SHA e HMAC	46
6	Criptografia Assimétrica — Fundamentos e RSA	47
7	Diffie-Hellman e Assinaturas Digitais	48
8	Criptografia de Curva Elíptica (ECC)	49
9	Infraestrutura de Chave Pública (PKI)	50
10	TLS e PGP — Esquemas Criptográficos Aplicados	51

Prefácio

Esta sebenta é um documento de estudo autónomo para o módulo de **Criptografia**, integrado na Unidade Curricular de Cibersegurança do Mestrado em Engenharia Eletrotécnica e de Computadores da Universidade de Trás-os-Montes e Alto Douro.

Organização da sebenta

Os capítulos correspondem às 10 aulas teóricas do módulo:

1. Criptografia Clássica — substituição, polialfabética e transposição
2. Cifras de Bloco — fundamentos, Feistel, DES e 3DES
3. AES — estágios e modos de operação
4. Cifras de Fluxo — RC4, Salsa20, ChaCha20 e padding
5. Hashing — propriedades, ataques, SHA e HMAC
6. Criptografia Assimétrica — fundamentos e RSA
7. Diffie-Hellman e Assinaturas Digitais
8. Criptografia de Curva Elíptica (ECC)
9. Infraestrutura de Chave Pública (PKI)
10. TLS e PGP — esquemas criptográficos aplicados

Autor

João Pavão — UTAD

1

Criptografia Clássica — Substituição, Polialfabética e Transposição

Introdução

A criptografia é a disciplina que estuda técnicas de protecção da informação através da sua transformação numa forma ilegível a quem não possua a chave para a reverter. Este capítulo introduz os conceitos fundamentais que atravessam toda a disciplina, o modelo de cifra simétrica, o princípio de Kerckhoffs, a distinção entre força bruta e criptanálise, e depois percorre as cifras clássicas: sistemas de substituição e de transposição desenvolvidos ao longo de séculos, antes do aparecimento do computador.

Estas cifras estão hoje completamente obsoletas do ponto de vista da segurança prática, sendo triviais de quebrar com os meios computacionais actuais. O seu valor neste módulo é pedagógico: introduzem, de forma simples e manual, os conceitos de chave, espaço de chaves, ataque por força bruta e criptanálise que se aplicam, com a mesma lógica, aos sistemas modernos estudados nos capítulos seguintes.

1.1 Fundamentos dos Sistemas Criptográficos

1.1.1 Classificação dos sistemas criptográficos

Os sistemas criptográficos classificam-se segundo três critérios independentes:

Critério	Categorias
Número de chaves	Simétrica (uma única chave, partilhada entre emissor e receptor) ou assimétrica (duas chaves distintas, uma pública e outra privada)
Tipo de operação	Substituição (cada símbolo é substituído por outro) ou transposição (os símbolos mantêm-se, mas a sua ordem é alterada)
Processamento do texto	Cifra de bloco (o texto é processado em blocos de tamanho fixo) ou cifra de fluxo (o texto é processado continuamente, símbolo a símbolo)

Este capítulo ocupa-se exclusivamente de sistemas **simétricos** e de **substituição** ou **transposição**. Os capítulos seguintes introduzem as cifras de bloco e de fluxo modernas.

1.1.2 O modelo de cifra simétrica

Um sistema criptográfico simétrico define-se por cinco componentes (Stallings 2020):

- **Texto claro** (X): a mensagem original, ainda inteligível.
- **Algoritmo de cifragem** (E): o procedimento que transforma o texto claro em texto cifrado, através de substituições e transformações sucessivas.
- **Chave secreta** (K): um valor independente do texto claro e do algoritmo, partilhado entre emissor e receptor por um canal seguro. Chaves diferentes produzem resultados diferentes para o mesmo texto claro e o mesmo algoritmo.
- **Texto cifrado** (Y): o resultado da cifragem, aparentemente aleatório e, à partida, ininteligível.
- **Algoritmo de decifragem** (D): essencialmente o algoritmo de cifragem em sentido inverso, que usa a mesma chave K para recuperar o texto claro a partir do texto cifrado.

Formalmente:

$$Y = E(K, X)$$

$$X = D(K, Y)$$

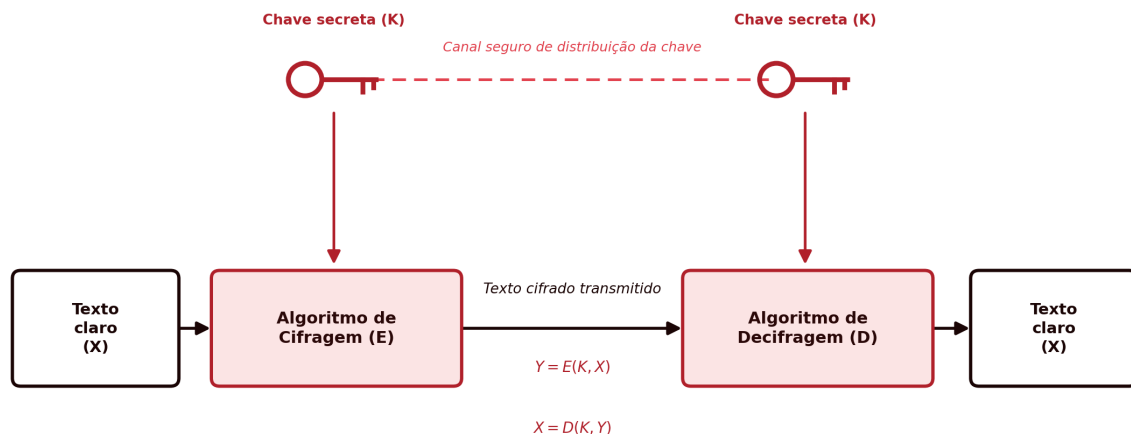


Figura 1.1: O modelo de cifra simétrica: emissor e receptor partilham previamente a mesma chave secreta, através de um canal seguro. O algoritmo de cifragem usa essa chave para transformar o texto claro em texto cifrado; o algoritmo de decifragem, usando a mesma chave, reverte a transformação.

Emissor e receptor têm de ter obtido cópias da mesma chave secreta por um meio seguro e mantê-la protegida (Figura 1.1). Se alguém descobrir a chave e conhecer o algoritmo, toda a comunicação cifrada com essa chave torna-se legível.

1.1.3 O princípio de Kerckhoffs

Um pressuposto fundamental de toda a criptografia moderna, e já presente nestas cifras clássicas, é o **princípio de Kerckhoffs**, formulado em 1883 (Kerckhoffs 1883): a segurança de

uma cifra deve depender apenas do segredo da chave, nunca do segredo do algoritmo. Assume-se sempre que um atacante conhece perfeitamente o algoritmo de cifragem e decifragem, apenas desconhecendo a chave.

Esta convenção tem uma consequência prática importante: um algoritmo cuja segurança dependa de permanecer secreto (*security through obscurity*) não é considerado seguro em criptografia séria, precisamente porque essa condição é impossível de garantir a longo prazo. Os algoritmos estudados neste módulo são todos públicos e amplamente escrutinados: a sua segurança assenta inteiramente no tamanho e no segredo da chave.

1.1.4 Força bruta e criptanálise

Um atacante que pretenda recuperar o texto claro sem conhecer a chave dispõe de duas estratégias gerais:

- **Força bruta:** testar exaustivamente todas as chaves possíveis, até encontrar uma que produza um texto claro inteligível. Em média, é necessário testar cerca de metade do espaço total de chaves para obter um resultado.
- **Criptanálise:** explorar conhecimento adicional sobre partes do sistema, como pares de texto claro e texto cifrado já conhecidos, ou propriedades estatísticas do algoritmo, para deduzir a chave sem a testar exaustivamente.

A criptanálise clássica sobre as cifras estudadas neste capítulo baseia-se sobretudo em **análise de frequência**: a distribuição estatística das letras numa língua natural não é uniforme, e essa assimetria sobrevive, de formas variáveis, à cifragem.

Consoante a informação disponível ao criptanalista, distinguem-se vários tipos de ataque (Stallings 2020):

Tipo de ataque	O que o criptanalista conhece
Apenas texto cifrado	O algoritmo e o texto cifrado
Texto claro conhecido	O algoritmo, o texto cifrado, e um ou mais pares texto claro/texto cifrado formados com a mesma chave
Texto claro escolhido	O algoritmo, o texto cifrado, e um texto claro escolhido pelo próprio atacante, com o correspondente texto cifrado
Texto cifrado escolhido	O algoritmo, o texto cifrado, e um texto cifrado escolhido pelo próprio atacante, com o correspondente texto decifrado
Texto escolhido	O conhecimento acumulado dos dois ataques anteriores

1.2 Cifras de Substituição

Numa cifra de substituição, cada símbolo do texto claro é **substituído** por outro símbolo, segundo uma regra fixa determinada pela chave. A ordem dos símbolos mantém-se inalterada; o que muda é a sua identidade.

1.2.1 A cifra de César

A cifra de César, atribuída a Júlio César, é a cifra de substituição mais simples: cada letra é substituída pela letra que se encontra k posições mais à frente no alfabeto, sendo k a chave. Por exemplo, com $k = 3$:

Texto claro: G U A R D A B E M O T E U S E G R E D O
 Texto cifrado: J X D U G D E H P R W H X V H J U H G R

Numerando as letras do alfabeto de 0 (A) a 25 (Z), a cifra de César define-se formalmente por:

$$C = E(k, p) = (p + k) \bmod 26$$

$$p = D(k, C) = (C - k) \bmod 26$$

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

A operação módulo 26 garante que o resultado se mantém sempre dentro do intervalo válido: quando a soma ultrapassa 25, o resto da divisão por 26 devolve-a ao alfabeto. Por exemplo, cifrando a letra “y” ($p = 24$) com $k = 3$: $C = (24 + 3) \bmod 26 = 27 \bmod 26 = 1$, que corresponde à letra “B”.

Espaço de chaves e força bruta. A chave k só pode tomar 25 valores distintos (excluindo $k = 0$, que não cifra nada, e $k = 26$, equivalente a $k = 0$). Um espaço de chaves tão reduzido torna a cifra de César trivialmente vulnerável a um ataque de força bruta: basta gerar as 25 hipóteses de texto decifrado e identificar a única que é inteligível.

Criptanálise por análise de frequência. Mesmo sem recorrer a força bruta, a cifra de César quebra-se facilmente por criptanálise, explorando duas fontes de informação: a estrutura do texto (os espaços entre palavras sobrevivem à cifragem, revelando o comprimento de cada palavra) e a língua do texto claro (permitindo procurar padrões de palavras curtas e frequentes, como “de”, “que”, “não”, “para”). Combinando estas pistas com a frequência relativa de cada letra, que numa língua natural está longe de ser uniforme, um criptanalista consegue tipicamente recuperar a chave em poucos minutos.

i Porque é tão fácil quebrar

A facilidade da criptanálise não está apenas no algoritmo em si, mas no facto de o atacante dispor de muito mais informação do que apenas o algoritmo e o texto cifrado: conhece a língua da mensagem e a sua estrutura (onde começam e acabam as palavras). Ocultar a estrutura, retirando os espaços do texto cifrado, dificulta o ataque manual, mas não o torna impossível: outros aspectos estatísticos da língua, como a frequência de pares de letras consecutivas, continuam disponíveis.

Exercício resolvido: criptanálise por dígrafos e trígrafos. Uma palavra curta que se repita na mensagem, como palavra separada e completa, produz sempre o mesmo bloco de letras no texto cifrado, já que a cifra de César aplica um deslocamento fixo. Considere-se o seguinte texto cifrado:

Texto cifrado: V X B L Z L V B C L U H V L V X B L Z L C L

O texto tem dois blocos que se repetem, cada um duas vezes: **XBL** e **ZL**. Em português, “QUE” é o trígrafo mais comum como palavra isolada, e “SE” um dos dígrafos mais comuns. Assumindo que XBL corresponde a “QUE” e ZL a “SE”:

$$k = (X - Q) \bmod 26 = (23 - 16) \bmod 26 = 7$$

$$k = (Z - S) \bmod 26 = (25 - 18) \bmod 26 = 7$$

As duas hipóteses concordam no mesmo $k = 7$, o que confirma a correspondência sem qualquer tentativa de força bruta. Note-se que é essencial que “QUE” e “SE” apareçam como palavras separadas, nunca embutidas dentro de outra palavra maior (por exemplo, “SE” dentro de “SEGUNDA” não contaria), já que só uma palavra isolada repetida garante um bloco de texto cifrado idêntico.

Com $k = 7$ conhecido, o resto do texto decifra-se de imediato, palavra a palavra, incluindo os dois blocos ainda por identificar (**VBCL** e **CL**, ambos curtos e fáceis de confirmar assim que se sabe o deslocamento):

Texto claro: O Q U E S E O U V E N A O E O Q U E S E V E

“O que se ouve não é o que se vê.”

💡 Exercício: quebrar sem ajuda

Aplicando a mesma técnica, quebre o texto cifrado seguinte (desta vez sem espaços entre palavras):

ZBFPDPGPYLZPZBFPDPZFGP

1.2.2 Cifra monoalfabética

A cifra de César é um caso particular, muito restrito, de uma classe mais geral: a **cifra monoalfabética**, em que a chave é uma permutação arbitrária das 26 letras do alfabeto, não apenas um deslocamento fixo. O espaço de chaves cresce de 25 para $26! \approx 4 \times 10^{26}$, tornando um ataque de força bruta computacionalmente inviável. Por exemplo:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Q	M	J	Z	T	G	F	K	P	W	L	S	B	O	X	N	C	R	Y	E	V	H	I	A	D	U

F V Q R Z Q M T B X E T V Y T F R T Z X

Apesar do espaço de chaves astronomicamente maior, a cifra monoalfabética continua vulnerável a criptanálise por análise de frequência: como cada letra do texto claro corresponde sempre à mesma letra no texto cifrado, a distribuição estatística das letras do texto claro transporta-se, intacta, para o texto cifrado, apenas com os símbolos trocados. Basta comparar a frequência das letras no texto cifrado com a frequência conhecida das letras na língua do texto claro para reconstruir grande parte da correspondência.

Em português, a distribuição das letras está longe de ser uniforme: A, E e O dominam claramente, seguidos por um grupo intermédio (S, R, I, N, D, M, U, T, C), e um conjunto de letras raras (Z, J, X e, sobretudo, K, W, Y, praticamente ausentes em palavras portuguesas). Os valores seguintes resultam de uma contagem real, sobre os três primeiros cantos d’*Os Lusíadas*, de Luís de Camões (cerca de 103 mil caracteres alfabéticos), com o alfabeto normalizado para a–z, sem acentuação nem cedilha. Serve aqui apenas de exemplo aproximado de recolhas em *corpus* maiores:

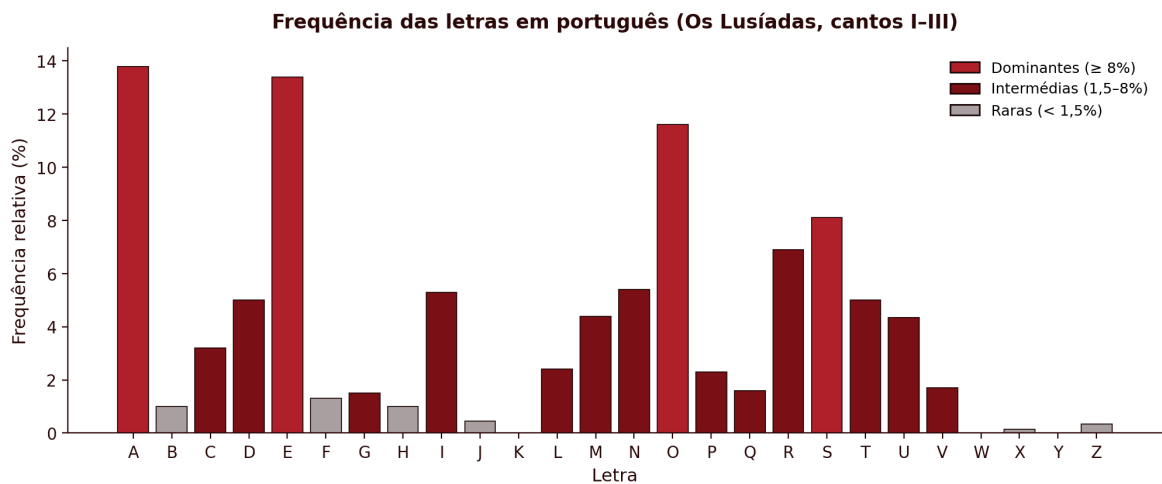


Figura 1.2: Frequência das letras em português, contada sobre os três primeiros cantos d’*Os Lusíadas*.

Contando as letras do texto cifrado do exemplo anterior (“FVQRZQ MTB X ETV YTFRTZX”, 20 letras):

Letra cifrada	Ocorrências	Observado
T	4	20,0%
F	2	10,0%
V	2	10,0%
Q	2	10,0%
R	2	10,0%
Z	2	10,0%
X	2	10,0%
M	1	5,0%
B	1	5,0%
E	1	5,0%
Y	1	5,0%

Comparando esta lista com o gráfico de referência (Figura 1.2): T seria o candidato óbvio a corresponder a A ou E (as letras dominantes da língua), mas seis letras diferentes empatam

Introdução

logo a seguir a 10%, sem nenhuma se destacar como o esperado segundo lugar. Com apenas 20 letras, cada ocorrência isolada vale 5 pontos percentuais, o que basta para distorcer completamente o quadro — a conclusão de que este texto é curto demais para uma análise de frequência séria não é uma suposição, é o que a própria tabela mostra. Considere-se, em vez disso, um texto real e bastante mais longo, cifrado com a mesma chave: a primeira estrofe de *Os Lusíadas*, de Luís de Camões (1572).

As armas e os barões assinalados,
Que da ocidental praia Lusitana,
Por mares nunca de antes navegados,
Passaram ainda além da Taprobana,
Em perigos e guerras esforçados,
Mais do que prometia a força humana,
E entre gente remota edificaram
Novo Reino, que tanto sublimaram.

Sem acentuação e sem espaços, as 221 letras deste excerto cifram-se, com a mesma chave usada acima, da seguinte forma:

```
QYQRBQYTXYMQRXTYQYYPOQSQZXY  
CVTZQXJPZTOEQSNRQPQSVYPEQOQ  
NXRBQRTYOVOJQZTQOETYQHTFQZXY  
NQYYQRQBQPOZQQSTBZQEQRXMQOQ  
TBNTRPFXYTFVTRRQYTYGXRJQZXY  
BQPYZXCVTNRXBTEPQQGXRJQKVBQOQ  
TTOERTFTOETRTBXEQTZPGPJQRQB  
OXHXRTPOXCVTEQOEXYVMSPBQRQB
```

Contando agora as letras deste texto cifrado (221 letras):

Letra cifrada	Ocorrências	Observado
Q	42	19,00%
T	25	11,31%
Y	19	8,60%
R	18	8,14%
X	18	8,14%
O	15	6,79%
B	12	5,43%
P	12	5,43%
Z	10	4,52%
E	10	4,52%
V	8	3,62%
N	6	2,71%
S	5	2,26%
J	5	2,26%

Ao contrário do exemplo curto, agora a ordem observada bate certo com a ordem de referência (Figura 1.2), posição a posição: o par dominante (Q, T) corresponde exactamente ao par dominante da língua (A, E); o grupo seguinte (Y, R, X, O, B, P, Z, E, V) corresponde, com pequenos desvios de ordem, ao grupo intermédio (S, R, I, N, D, M, U, T, C). Um criptanalista sem acesso ao texto claro, só com esta tabela e o gráfico de referência, teria motivos sólidos para arriscar $Q \leftrightarrow A$ e $T \leftrightarrow E$ logo à partida, e resolveria a maior parte da chave por este processo, palavra a palavra, tal como se fez atrás com “QUE” e “SE”. A diferença essencial em relação ao exemplo de 20 letras não é o método, é a fiabilidade dos números que o método

recebe como ponto de partida. Esta é uma criptanálise baseada em estatística, quanto maior for a mensagem mais acertivo é o processo.

1.2.3 Cifra de Playfair

A cifra de Playfair reduz o problema anterior ao cifrar **pares de letras** (dígrafos), não letras isoladas, usando uma grelha 5×5 construída a partir de uma palavra-chave. Ao cifrar pares em vez de letras individuais, a distribuição de frequências de letras isoladas deixa de se transportar directamente para o texto cifrado, dificultando a análise de frequência simples.

Construção da grelha. Escolhe-se uma palavra-chave, escrevem-se as suas letras (sem repetições) nas primeiras posições da grelha, e completam-se as posições restantes com as letras do alfabeto por ordem, saltando as já usadas. Como a grelha só tem 25 posições para 26 letras, “I” e “J” partilham a mesma célula. Com a palavra-chave “REDIAL” (uma aldeia do concelho de Murça):

R	E	D	I/J	A
L	B	C	F	G
H	K	M	N	O
P	Q	S	T	U
V	W	X	Y	Z

Figura 1.3: Grelha de Playfair construída a partir da palavra-chave “REDIAL”.

Regras de cifragem, aplicadas a cada par de letras do texto claro:

- Se as duas letras estão na **mesma linha**, cada uma é substituída pela letra imediatamente à sua direita (voltando ao início da linha, se necessário).
- Se as duas letras estão na **mesma coluna**, cada uma é substituída pela letra imediatamente abaixo (voltando ao topo, se necessário).
- Caso contrário, cada letra é substituída pela letra que está na sua própria linha, mas na coluna da outra letra (as duas letras e as suas substitutas formam um rectângulo na grelha).

Se as duas letras de um par forem iguais, ou se sobrar uma letra no fim de um texto com número ímpar de letras, insere-se uma letra de enchimento (aqui, “X”).

Considere-se a frase “UTAD É FIXE”. Sem espaços nem acentos, e dividida em pares (a última letra, sozinha, recebe um “X” de enchimento):

UT AD EF IX EX

Aplicando as regras: “UT” está na mesma linha (U e T na 4.^a linha) → PU. “AD” está na mesma linha (a 1.^a) → RI. “EF” não partilha linha nem coluna → forma-se o rectângulo, dando IB. “IX” e “EX”, também sem linha nem coluna comuns, dão DY e DW, respectivamente. O texto cifrado final:

PURIIBDYDW

Ainda assim, a Playfair permanece vulnerável: a distribuição de frequências dos pares de letras (dígrafos) numa língua natural também não é uniforme, e essa assimetria sobrevive à cifra de forma análoga à das letras isoladas. Um criptanalista com texto cifrado suficiente consegue reconstruir a grelha através de análise de frequência de dígrafos.

1.2.4 Cifra polialfabética: Vigenère

As cifras anteriores partilham uma fragilidade estrutural: cada letra do texto claro corresponde sempre à mesma letra (ou par de letras) no texto cifrado, ao longo de toda a mensagem. As **cifras polialfabéticas** eliminam esta correspondência fixa, usando várias substituições diferentes ao longo do texto, alternadas segundo uma palavra-chave que se repete ciclicamente.

Na cifra de Vigenère, a palavra-chave define uma sequência de deslocamentos (cada letra da chave corresponde a um deslocamento de César diferente), aplicados ciclicamente às sucessivas letras do texto claro. Sendo m o comprimento da palavra-chave, e $k_0, k_1, \dots, k_{m-1}$ os deslocamentos que ela define:

$$C_i = (p_i + k_{i \bmod m}) \bmod 26 \quad p_i = (C_i - k_{i \bmod m}) \bmod 26$$

Como a chave é mais curta do que a mensagem (m é tipicamente muito menor do que o número de letras do texto claro), o índice $i \bmod m$ garante que ela se repete ciclicamente até cobrir todo o texto. A mesma letra do texto claro pode assim corresponder a letras diferentes no texto cifrado, consoante a posição em que ocorre, o que achata consideravelmente a distribuição de frequências observável no texto cifrado.

Usando de novo a chave “REDIAL”, repetida tantas vezes quantas as necessárias para cobrir todo o texto claro, e o trava-línguas “Três tigres tristes dormem”:

```
Chave:      R E D I A L R E D I A L R E D I A L R E D I A
Texto claro: T R E S T I G R E S T R I S T E S D O R M E M
Texto cifrado: K V H A T T X V H A T C Z W W M S O F V P M M
```

Repare-se no bloco **VHA**, que aparece duas vezes no texto cifrado. Não é uma palavra repetida desta vez, é uma coincidência genuína do próprio trava-línguas: a sequência “RES” surge em “**TRES**” e, por acaso, também em “**ti**g**RES**”, exatamente 6 posições depois. Como a chave “REDIAL” também tem 6 letras, as duas ocorrências de “RES” alinham-se com a chave da mesma forma, produzindo o mesmo bloco cifrado. Este é o princípio por trás do

exame de Kasiski: sempre que um trecho do texto claro se repete a uma distância múltipla do comprimento da chave, mesmo por coincidência, o texto cifrado correspondente repete-se também, e essa repetição visível é uma pista directa sobre o comprimento da chave — aqui, 6.

A cifra de Vigenère resistiu a criptanálise sistemática durante séculos, mas não é inquebrável: se a chave for curta e repetida muitas vezes ao longo de um texto suficientemente longo, técnicas como o exame de Kasiski (identificação de repetições no texto cifrado, que revelam múltiplos do comprimento da chave) permitem recuperar o comprimento da chave e, a partir daí, tratar o texto como várias cifras de César independentes, uma por posição dentro do ciclo da chave.

Sabendo já, só pelo exame de Kasiski, que a chave tem 6 letras (sem ainda saber quais), o criptanalista reagrupa o texto cifrado: todas as letras nas posições 1, 7, 13, 19, ... formam um primeiro grupo; as posições 2, 8, 14, 20, ... formam um segundo grupo; e assim sucessivamente até ao sexto grupo. Cada grupo, tomado isoladamente, é uma cifra de César comum, com um único k fixo, exactamente o tipo de problema já resolvido na secção anterior:

```
Grupo 1 (pos. 1, 7, 13, 19): K X Z F
Grupo 2 (pos. 2, 8, 14, 20): V V W V
Grupo 3 (pos. 3, 9, 15, 21): H H W P
Grupo 4 (pos. 4, 10, 16, 22): A A M M
Grupo 5 (pos. 5, 11, 17, 23): T T S M
Grupo 6 (pos. 6, 12, 18):    T C O
```

Note-se que isto não contradiz o que se disse atrás sobre a Vigenère achatar a distribuição de frequências: essa distorção afecta o texto cifrado *como um todo*, onde letras cifradas com deslocamentos diferentes se misturam. Dentro de cada grupo isolado, porém, foi sempre usado o mesmo deslocamento, pelo que a distribuição volta a ser a de uma César normal, sem achatamento nenhum. O exame de Kasiski e a análise de frequência resolvem, por isso, dois problemas diferentes e complementares: um dá o número de grupos a formar (o comprimento da chave); o outro, aplicado depois a cada grupo separadamente, dá o deslocamento de cada um.

A cada grupo aplica-se, isoladamente, a mesma análise de frequência de letra única usada contra a cifra de César: qual a letra mais frequente deste grupo, e a que letra da língua deverá corresponder? Neste exemplo, com apenas 3 a 4 letras por grupo, não há dados suficientes para essa análise ser fiável, tal como já se viu antes com textos curtos; num texto real, bastante mais longo, cada grupo teria dezenas ou centenas de letras, e o método funciona tão bem quanto funcionou contra a César.

1.2.5 Cifra de Vernam

Antes de se chegar à solução definitiva, vale a pena um passo intermédio. A **cifra de Vernam**, criada por Gilbert Vernam nos laboratórios AT&T em 1918 para telegrafia (Stallings 2020), generaliza a Vigenère de duas formas: substitui a soma modular por uma operação **XOR** ao nível dos bits, e usa uma chave muito mais longa do que a palavra-chave curta típica da Vigenère, ao ponto de, na prática, ser tão longa quanto a própria fita de papel perfurado que a implementava.

$$C_i = P_i \oplus K_{i \bmod m} \quad P_i = C_i \oplus K_{i \bmod m}$$



O texto claro não é "cifrado" por nenhum algoritmo: é apenas combinado bit a bit com a chave por XOR ($\oplus$). Toda a segurança está na imprevisibilidade da chave.

Figura 1.4: A cifra de Vernam: um gerador de fluxo de chave produz a sequência k_i , combinada por XOR com o texto claro na cifragem, e com o texto cifrado na decifragem — a mesma operação em ambos os sentidos.

Uma chave muito mais longa dificulta consideravelmente o exame de Kasiski, já que as repetições no texto cifrado tornam-se muito mais raras. Mas o problema de fundo não desaparece: mais cedo ou mais tarde, mesmo uma fita de chave muito longa esgota-se e tem de recomeçar, voltando a introduzir um ciclo, ainda que muito maior do que o de uma palavra-chave Vigenère. A cifra de Vernam é, por isso, uma melhoria substancial sobre a Vigenère, mas não uma solução definitiva: continua vulnerável, em princípio, ao mesmo tipo de criptanálise, apenas exigindo mensagens interceptadas muito mais longas para se tornar praticável.

Note-se uma limitação estrutural da cifra de Vernam, tal como concebida originalmente: usa apenas uma chave, sem qualquer outro valor que distinga uma mensagem da seguinte. É precisamente essa ausência que obriga a fita de chave a repetir-se assim que se esgota. As cifras de fluxo modernas, retomadas no capítulo 4, resolvem este problema introduzindo um segundo valor, o *nonce*, único por mensagem mas não secreto, que permite reutilizar a mesma chave em segurança de mensagem para mensagem, sem nunca repetir o fluxo de chave.

1.2.6 One-Time Pad

O **One-Time Pad** (cifra de utilização única) resolve definitivamente a fragilidade que a cifra de Vernam apenas atenuava: usa uma chave verdadeiramente aleatória, do mesmo comprimento que a mensagem, nunca reutilizada noutra mensagem, e por isso nunca sujeita a ciclo nenhum. Cada letra do texto claro é combinada com a letra correspondente da chave, tal como no Vigenère, mas sem qualquer ciclo, uma vez que a chave é tão longa quanto o próprio texto.

Claude Shannon demonstrou, em 1949, que o One-Time Pad, quando usado correctamente, é **matematicamente inquebrável**: o texto cifrado não revela nenhuma informação sobre o texto claro, mesmo com poder computacional ilimitado (Shannon 1949). Esta propriedade chama-se **segredo perfeito**.

Um exemplo concreto de segredo perfeito. Para qualquer texto cifrado C e qualquer texto claro P do mesmo comprimento, existe sempre exactamente uma chave $K = (C - P) \bmod 26$

Introdução

que os liga. Como todas as chaves são, à partida, igualmente prováveis, o texto cifrado não favorece nenhuma leitura em detrimento de outra: para um mesmo texto cifrado, existe sempre uma chave que o decifra em qualquer texto claro que se queira, do mesmo comprimento.

Isto demonstra-se de forma concreta. Fixando um único texto cifrado de 16 letras, encontram-se três chaves diferentes que o decifram em três frases portuguesas plausíveis, sobre o mesmo assunto e mutuamente incompatíveis entre si:

```
Texto cifrado (fixo):           Q X B Z R P P N M N R S X S N J

Chave 1: Q F X M K P L V T N Z O R Y W J
Decifra para: A SENHA ESTA SEGURA

Chave 2: Q V U Z W L K Z E W D Y W S K J
Decifra para: A CHAVE FOI ROUBADA

Chave 3: C E N P N C C T Z L R G D P Z P
Decifra para: O TOKEN NUNCA MUDOU
```

As três decifragens são correctas: cada uma resulta de aplicar $p = (C - K) \bmod 26$, letra a letra, com a respectiva chave. Um atacante que intercepte apenas QXBZRPPNMNRSXSNJ, sem conhecer a chave, não tem forma alguma de saber se a senha está segura, foi roubada, ou se, afinal, o assunto era antes o token, e nunca a senha. As três leituras, e infinitas outras do mesmo comprimento, são igualmente compatíveis com o texto cifrado.

O mesmo efeito reproduz-se em qualquer língua. Com um texto cifrado diferente, de 19 letras, obtêm-se três leituras em inglês, também mutuamente incompatíveis:

```
Ciphertext (fixed):           J O B P D C R H E U G R W H U X Z G H

Key 1: X Q M P L K V T N R O Y W J C F Z B D
Decrypts to: MY PASSWORD STAYS SAFE

Key 2: Q H T X L Y P Q A B A D D P B J O C U
Decrypts to: THIS SECRET GOT STOLEN

Key 3: Q H T X D N J X A W K R E P N X I C E
Decrypts to: THIS API KEY WAS SHARED
```

Apesar desta garantia teórica absoluta, o One-Time Pad é impraticável na generalidade dos cenários reais, por três exigências difíceis de satisfazer em simultâneo:

- a chave tem de ser tão longa quanto a mensagem;
- a chave tem de ser genuinamente aleatória, não gerada por um algoritmo determinístico;
- a chave nunca pode ser reutilizada, nem parcialmente.

Distribuir e armazenar chaves deste tamanho, de forma segura, é tão difícil quanto distribuir a própria mensagem em segurança. A isto acresce um custo menos óbvio: gerar, de forma genuinamente aleatória, e depois gerir com segurança um grande volume de chaves desta dimensão tem também um custo computacional apreciável, que pode tornar o processo lento

de mais para a generalidade das aplicações práticas. Estas exigências combinadas limitam o uso do One-Time Pad a cenários muito específicos, como certas comunicações diplomáticas ou militares de importância excepcional.

1.3 Cifras de Transposição

Ao contrário das cifras de substituição, as **cifras de transposição** não alteram a identidade dos símbolos do texto claro, apenas a sua ordem. O texto cifrado contém exactamente as mesmas letras do texto claro, apenas reorganizadas segundo uma regra determinada pela chave.

1.3.1 A técnica rail fence

A forma mais simples de transposição é a técnica **rail fence** (cerca de caminho de ferro): o texto claro escreve-se alternadamente em duas linhas, formando um ziguezague, e o texto cifrado obtém-se lendo primeiro a linha de cima, depois a linha de baixo, sem interrupções.

Para o texto claro “ESCONDE BEM A CHAVE” (sem espaços, 16 letras):

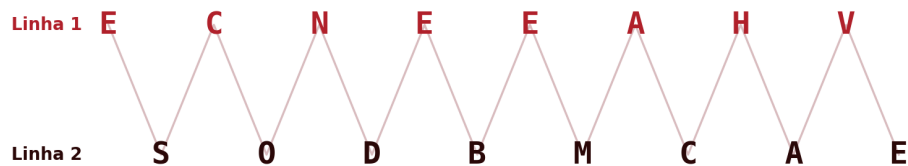


Figura 1.5: A técnica rail fence: o texto claro escreve-se em ziguezague por duas linhas; o texto cifrado lê-se linha a linha.

Lendo a Linha 1 seguida da Linha 2, obtém-se o texto cifrado ECNEEAHVSO DBMCAE. Todas as letras do texto cifrado pertencem ao texto claro, apenas reordenadas, pelo que a criptanálise estatística sobre a frequência de letras isoladas continua perfeitamente viável, tal como se discutirá adiante nesta secção.

1.3.2 Transposição colunar

Uma técnica mais elaborada é a **transposição colunar**: o texto claro é escrito numa grelha com um número de colunas igual ao comprimento da palavra-chave, e o texto cifrado obtém-se lendo as colunas pela ordem alfabética das letras da palavra-chave, em vez da ordem original em que foram escritas.

Usando de novo a palavra-chave “REDIAL” — agora não para deslocar nem substituir, mas para ordenar colunas — e a frase “Isto fica só entre nós” (18 letras, exactamente três linhas de seis colunas, sem necessidade de enchimento). A cada letra da chave associa-se a sua posição na ordem alfabética (A=1, D=2, E=3, I=4, L=5, R=6), e o texto claro escreve-se por linhas, coluna a coluna, por baixo da chave:

R (6)	E (3)	D (2)	I (4)	A (1)	L (5)
I	S	T	O	F	I
C	A	S	O	E	N
T	R	E	N	O	S

Figura 1.6: Grelha de transposição colunar com a chave “REDIAL”. O número entre parêntesis, por baixo de cada letra da chave, indica a ordem alfabética dessa letra e, por isso, a ordem de leitura da respectiva coluna.

O texto cifrado obtém-se lendo as colunas pela ordem numérica das suas posições (não pela ordem em que aparecem na grelha): primeiro a coluna sob “A” (posição 1), depois a coluna sob “D” (posição 2), e assim sucessivamente até à coluna sob “R” (posição 6):

FEO TSE SAR OON INS ICT

Ou seja, “FEOTSESAROONINSICT”. Note-se que todas as letras do texto claro continuam presentes no texto cifrado, na mesma quantidade; só a ordem mudou.

Uma única transposição colunar é vulnerável a criptanálise, uma vez que a distribuição de frequência de letras isoladas se mantém inteiramente inalterada, apenas a sua posição relativa muda. Há aqui uma diferença importante em relação à substituição: numa substituição, a frequência serve para adivinhar que letra do texto claro corresponde a cada letra do texto cifrado, um problema de mapeamento; numa transposição, não há mapeamento nenhum para adivinhar, já que uma letra “E” no texto cifrado é mesmo um “E” do texto original, apenas deslocado de posição. A letra mais frequente do texto cifrado é, por isso, directamente a letra mais frequente da língua, sem qualquer hipótese intermédia — o que permite, inclusive, **distinguir** uma transposição de uma substituição só pela frequência: se a letra mais frequente do texto cifrado coincide com a esperada para a língua (A ou E, em português), é sinal de transposição, não de substituição. O que a frequência de letras isoladas não revela, sozinha, é a ordem de leitura das colunas: para isso são necessárias outras pistas, como dígrafos e trígrafos frequentes partidos ao meio pela grelha, ou testar várias larguras de coluna. Repetir a transposição várias vezes, com chaves diferentes (**transposição dupla** ou múltipla), dificulta consideravelmente a reconstrução manual, ainda que continue vulnerável a ataques computacionais modernos.

Historicamente, a combinação de substituição e transposição, cada uma delas fraca isoladamente, mas fortes quando combinadas, é o princípio que dá origem aos conceitos modernos de **confusão** e **difusão**, introduzidos por Shannon e centrais no desenho das cifras de bloco modernas estudadas no capítulo seguinte.

1.4 Síntese do Capítulo

Este capítulo estabeleceu o vocabulário e os princípios fundamentais que atravessam toda a disciplina de criptografia:

1. Um sistema criptográfico simétrico define-se pelos seus cinco componentes, texto claro, algoritmo de cifragem, chave secreta, texto cifrado e algoritmo de decifragem, relacionados por $Y = E(K, X)$ e $X = D(K, Y)$
2. O **princípio de Kerckhoffs** estabelece que a segurança de uma cifra deve depender apenas do segredo da chave, nunca do segredo do algoritmo
3. Um atacante dispõe de duas estratégias gerais: **força bruta** (testar exaustivamente o espaço de chaves) e **criptanálise** (explorar informação adicional para deduzir a chave sem a testar exaustivamente)
4. As **cifras de substituição** (César, monoalfabética, Playfair, Vigenère) substituem símbolos do texto claro por outros símbolos, com vulnerabilidades decrescentes, mas nunca totalmente eliminadas, a ataques de análise de frequência
5. A **cifra de Vernam** generaliza a Vigenère com XOR e uma chave muito mais longa, atenuando mas não eliminando o problema da chave que se repete; o **One-Time Pad** elimina-o por completo, sendo a única cifra clássica matematicamente inquebrável, ao custo de exigências de gestão de chaves impraticáveis na generalidade dos cenários reais
6. As **cifras de transposição** reordenam os símbolos do texto claro sem alterar a sua identidade, sendo a base histórica dos conceitos modernos de confusão e difusão

1.5 Para Pensar

Porque é que o One-Time Pad, apesar de matematicamente inquebrável, não é a solução universal para todos os problemas de comunicação segura? Que outro problema, tão difícil quanto o da própria mensagem, a sua utilização acaba por introduzir?

Referências

- AlFardan, Nadhem J., Daniel J. Bernstein, Kenneth G. Paterson, Bertram Poettering, e Jacob C. N. Schuldt. 2013. «On the Security of RC4 in TLS». Em *22nd USENIX Security Symposium*.
- Aumasson, Jean-Philippe. 2017. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press.
- Böck, Hanno, Aaron Zauner, Sean Devlin, Juraj Somorovsky, e Philipp Jovanovic. 2016. «Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS». Em *USENIX Workshop on Offensive Technologies (WOOT)*.
- Du, Wenliang. 2022. *Computer Security: A Hands-on Approach*. 3.^a ed. CreateSpace / Self-published.
- Dworkin, Morris. 2001. «Recommendation for Block Cipher Modes of Operation: Methods and Techniques». SP 800-38A. NIST.
- . 2007. «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC». SP 800-38D. NIST.
- Fluhrer, Scott, Itsik Mantin, e Adi Shamir. 2001. «Weaknesses in the Key Scheduling Algorithm of RC4». Em *Selected Areas in Cryptography (SAC 2001)*.
- Kerckhoffs, Auguste. 1883. «La cryptographie militaire». *Journal des Sciences Militaires* 9: 5–38.

Referências

- Martin, Keith. 2017. *Everyday Cryptography*. 2.^a ed. Oxford University Press.
- National Institute of Standards and Technology. 2001. «Advanced Encryption Standard (AES)». FIPS PUB 197. NIST.
- Nir, Yoav, e Adam Langley. 2018. «ChaCha20 and Poly1305 for IETF Protocols». RFC 8439. IETF.
- Popov, Andrei. 2015. «Prohibiting RC4 Cipher Suites». RFC 7465. IETF.
- Shannon, Claude E. 1949. «Communication Theory of Secrecy Systems». *Bell System Technical Journal* 28 (4): 656–715.
- Stallings, William. 2020. *Cryptography and Network Security: Principles and Practice*. 8.^a ed. Pearson.

2

Cifras de Bloco — Fundamentos, Feistel, DES e 3DES

Introdução

As cifras clássicas estudadas no capítulo anterior operam sobre letras de um alfabeto natural: substituem-nas, transpõem-nas, mas trabalham sempre com a estrutura de uma língua. Um computador não lê letras; lê sequências de bits. As cifras modernas partem deste facto: o texto claro, seja ele texto, imagem ou som, é sempre, em última análise, uma sequência binária, e é sobre essa sequência que as cifras de bloco operam.

Este capítulo introduz a estrutura comum à generalidade das cifras de bloco modernas, a rede de Feistel, e dois algoritmos históricos construídos sobre ela: o DES, durante décadas o padrão dominante, e o 3DES, a extensão criada para prolongar a sua vida útil depois de o DES ter sido considerado inseguro. O capítulo seguinte trata do AES, o algoritmo que hoje ocupa o lugar que o DES ocupou.

2.1 Fundamentos das Cifras de Bloco

2.1.1 O que é uma cifra de bloco

Uma **cifra de bloco** divide o texto claro em blocos de tamanho fixo, tipicamente 64 ou 128 bits, e cifra cada bloco independentemente, usando sempre a mesma chave. Formalmente, para um bloco de n bits e uma chave K , o algoritmo de cifragem E_K é uma função bijectiva:

$$E_K : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

Ou seja, para cada chave K , E_K é uma permutação distinta do conjunto de todos os blocos possíveis: cada bloco de texto claro tem exactamente um bloco de texto cifrado correspondente, e vice-versa, o que garante que a decifragem é sempre possível e única.

Isto generaliza directamente a cifra monoalfabética do capítulo anterior, que também era uma permutação, mas do alfabeto de 26 letras. Uma cifra de bloco com blocos de 64 bits é uma permutação de um conjunto com 2^{64} elementos, um espaço imensamente maior, o que por si só já dificulta a análise de frequência: em vez de 26 símbolos possíveis, há 2^{64} blocos possíveis, cada um ocorrendo raramente o suficiente para que contar frequências deixe de ser útil.

2.1.2 A cifra ideal, e porque é impraticável

Se uma cifra de bloco é, no fundo, uma permutação parametrizada pela chave, a construção mais directamente segura seria escolher, para cada chave, uma permutação **completamente arbitrária** entre todos os blocos de entrada e todos os blocos de saída, sem qualquer estrutura interna que um criptanalista pudesse explorar. A esta construção chama-se a **cifra ideal**, e o problema com ela não é de segurança, é de tamanho da chave.

Formalmente, o algoritmo de cifragem E_K de uma cifra de bloco de n bits é uma função

$$E_K : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

que mapeia cada um dos 2^n blocos de entrada possíveis a um bloco de saída, também de n bits. Considere-se o caso $n = 4$: cada bloco de entrada é um valor entre 0000 e 1111, mais convenientemente escrito em **hexadecimal**, já que um único algarismo hexadecimal representa exactamente 4 bits. Os 16 blocos possíveis correspondem, um a um, aos algarismos 0 a F.

Uma **permutação arbitrária** destes 16 valores é qualquer reatribuição bijectiva de cada entrada a uma saída distinta, sem repetições. Por exemplo, estas três permutações são todas válidas, e completamente diferentes entre si:

Entrada:	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Permutação A (+5):	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4
Permutação B (livre):	5	D	2	9	0	B	7	E	3	F	1	8	6	4	A	C
Permutação C (pares):	0	1	3	2	5	4	7	6	9	8	B	A	D	C	F	E

A Permutação A é apenas um deslocamento — a mesma lógica da cifra de César, agora sobre hexadecimais em vez de letras. As permutações B e C não seguem nenhuma fórmula simples: foram escolhidas valor a valor, sem qualquer padrão, apenas garantindo que cada entrada tem uma saída diferente e que nenhuma saída se repete.

Quantas permutações destas existem, ao todo? Para a primeira entrada (0), há 16 saídas possíveis; escolhida essa, sobram 15 opções para a entrada seguinte (1), já que a saída escolhida para 0 não pode repetir-se; depois 14 para a entrada 2; e assim sucessivamente, até restar apenas 1 opção para a última entrada (F). O total é:

$$16 \times 15 \times 14 \times \dots \times 1 = 16! \approx 2,09 \times 10^{13}$$

Este número não depende em nada do tamanho da chave: é apenas a contagem de todas as formas de baralhar 16 valores, quer esses valores sejam algarismos hexadecimais, cartas de um baralho, ou letras de um alfabeto.

É aqui que entra o problema da chave. Se a chave K tivesse também 4 bits, existiriam apenas 16 valores de chave possíveis — e cada chave, através do algoritmo, só consegue determinar *uma* permutação. Por muito engenhoso que seja o algoritmo, 16 chaves nunca conseguem seleccionar mais do que 16 permutações diferentes, de um total de $16! \approx 2,09 \times 10^{13}$: por exemplo, a chave 5 podia seleccionar a Permutação A, a chave 3 a B, e assim por diante, mas ficam sempre só 16 possibilidades cobertas. A cifra de César é exactamente este caso: as suas 16 chaves seleccionam sempre 16 deslocamentos (como a Permutação A), nunca uma

permutação livre como a B ou a C. Para a chave conseguir alcançar **qualquer uma** das $16!$ permutações, e não apenas um pequeno subconjunto escolhido à partida, precisa de ter, ela própria, muitos mais de 4 bits.

Quão maior, exactamente? Há duas respostas para esta pergunta: a primeira dá o tamanho REAL da chave para endereçar todas as permutações possíveis de blocos de 4 bits para blocos de 4 bits; a segunda é uma estimativa que maximiza esse valor.

Sabendo que há $16!$ permutações válidas o número de bits, x necessário para mapear esse número é o tamanho da chave. Então $2^x = (16!)$ o que nos dá $x = \log_2(16!) \approx 45$ bits.

A segunda maneira dá uma forma mais directa de responder ao tamanho da chave, que é imaginar que esta *é*, literalmente, a tabela de substituição completa. Por exemplo, para a Permutação A (a de “+5”) têm-se a seguinte tabela:

Entrada	Saída (4 bits)	Saída (Hex.)
0	→ 0 1 0 1	(= 5)
1	→ 0 1 1 0	(= 6)
2	→ 0 1 1 1	(= 7)
3	→ 1 0 0 0	(= 8)
4	→ 1 0 0 1	(= 9)
5	→ 1 0 1 0	(= A)
6	→ 1 0 1 1	(= B)
7	→ 1 1 0 0	(= C)
8	→ 1 1 0 1	(= D)
9	→ 1 1 1 0	(= E)
A	→ 1 1 1 1	(= F)
B	→ 0 0 0 0	(= 0)
C	→ 0 0 0 1	(= 1)
D	→ 0 0 1 0	(= 2)
E	→ 0 0 1 1	(= 3)
F	→ 0 1 0 0	(= 4)

Da tabela vemos que o tamanho da chave tem que ser tal que possa endereçar uma *string* numérica de 16 caracteres hexadecimais. Portanto tem que ter o tamanho de 16 caracteres. Como cada caractere tem 4 bits, o tamanho da chave em bits é $4 \times 16 = 64$ o que corresponde à área da tabela mostrada:

$$\text{altura} \times \text{largura} = 16 \times 4 = 64 \text{ bits}$$

Generalizando, para um bloco de n bits, a chave da cifra ideal precisa de

$$n \times 2^n \text{ bits}$$

Para $n = 4$, isto são apenas 64 bits, perfeitamente gerível. Mas para um bloco de $n = 64$ bits, um tamanho perfeitamente razoável para uma cifra real:

$$64 \times 2^{64} = 2^6 \times 2^{64} = 2^{70} \approx 10^{21} \text{ bits}$$

Um número sem qualquer aplicação prática: nenhum sistema consegue gerar, armazenar, distribuir ou sequer escrever uma chave desta dimensão. **A cifra ideal é, por isso, impraticável.** As cifras de bloco reais não tentam realizá-la directamente; em vez disso,

procuram **aproximar-se** do seu comportamento imprevisível, combinando repetidamente, ao longo de várias rondas, operações simples e eficientes de especificar, até que o resultado final seja computacionalmente indistinguível de uma permutação verdadeiramente arbitrária. É exactamente para isto que servem a confusão e a difusão, apresentadas de seguida.

2.1.3 Confusão e difusão

Claude Shannon, o mesmo autor que demonstrou o segredo perfeito do One-Time Pad, identificou em 1949 duas propriedades que uma cifra robusta deve garantir:

- **Confusão:** a relação entre a chave e o texto cifrado deve ser o mais complexa e não linear possível, de modo a esconder qualquer padrão estatístico que ligue os dois. Na prática, obtém-se através de operações de substituição não lineares.
- **Difusão:** a influência de cada bit do texto claro (e de cada bit da chave) deve espalhar-se por muitos bits do texto cifrado, de forma que alterar um único bit de entrada altere, em média, metade dos bits de saída, de forma aparentemente aleatória. Na prática, obtém-se através de operações de permutação e mistura de bits.

Nenhuma das duas propriedades, isoladamente, é suficiente: a substituição sozinha, como se viu na cifra monoalfabética, é vulnerável à frequência; a transposição sozinha mantém a frequência inteiramente intacta. As cifras de bloco modernas combinam as duas, aplicando repetidamente, ao longo de várias **rondas**, uma sequência de substituições e permutações, até que o texto cifrado final não deixe transparecer nenhuma relação estatística útil com o texto claro ou com a chave.

2.2 A Rede de Feistel

A generalidade das cifras de bloco clássicas, incluindo o DES, organiza estas rondas segundo uma estrutura proposta por Horst Feistel, nos anos 1970, enquanto trabalhava na IBM. Numa **rede de Feistel**, o bloco divide-se em duas metades iguais, L_i (esquerda) e R_i (direita), e cada ronda i aplica-se assim:

$$\begin{aligned}L_{i+1} &= R_i \\ R_{i+1} &= L_i \oplus F(R_i, K_i)\end{aligned}$$

onde F é uma função de ronda, tipicamente não linear, parametrizada por uma subchave K_i derivada da chave principal, e $\oplus$ representa a operação XOR bit a bit.

A propriedade fundamental de uma rede de Feistel é que a decifragem usa exactamente a mesma estrutura da cifragem, apenas com as subchaves aplicadas em ordem inversa, e isto é verdade **independentemente da função F escolhida**, mesmo que F não seja, em si, invertível. Isto simplifica consideravelmente a implementação: o mesmo circuito ou código serve para cifrar e para decifrar, o que não seria necessariamente possível caso F tivesse de ser tanto aplicada como invertida.

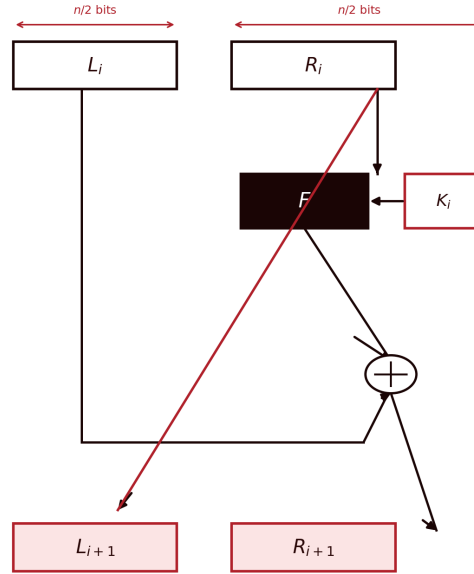


Figura 2.1: Uma rodada de uma rede de Feistel, exemplificada para um bloco de $W = 64$ bits (a notação LE_i/RE_i usada na figura é equivalente a L_i/R_i). A metade direita RE_0 atravessa uma função F parametrizada pela subchave da ronda K_1 , o resultado combina-se por XOR com a metade esquerda LE_0 , e as duas metades trocam de posição para a ronda seguinte (LE_1/RE_1).

2.2.1 Exemplo com uma rede reduzida

Para tornar isto concreto, considere-se uma versão drasticamente reduzida, com blocos de apenas 8 bits (metades de 4 bits) e a função $F(R, K) = \text{ROL}_1(R) \oplus K$, onde ROL_1 roda os 4 bits de R um lugar para a esquerda. Esta função não tem qualquer pretensão de segurança real, serve apenas para ilustrar o mecanismo.

Com texto claro $L_0 = 1010$, $R_0 = 0110$, e duas subchaves $K_1 = 0011$, $K_2 = 1100$:

Ronda 1: $F(R_0, K_1) = \text{ROL}_1(0110) \oplus 0011 = 1100 \oplus 0011 = 1111$ $L_1 = R_0 = 0110$ $R_1 = L_0 \oplus F(R_0, K_1) = 1010 \oplus 1111 = 0101$

Ronda 2: $F(R_1, K_2) = \text{ROL}_1(0101) \oplus 1100 = 1010 \oplus 1100 = 0110$ $L_2 = R_1 = 0101$ $R_2 = L_1 \oplus F(R_1, K_2) = 0110 \oplus 0110 = 0000$

O texto cifrado é $L_2R_2 = 01010000$. Para decifrar, aplicam-se as mesmas duas rondas, mas com as subchaves por ordem inversa (K_2 primeiro, depois K_1), recuperando exactamente $L_0 = 1010$, $R_0 = 0110$ — confirmado por computação directa. Repare-se que em nenhum momento foi necessário inverter a função F : só a ordem das subchaves mudou.

2.3 DES — Data Encryption Standard

O **DES** (*Data Encryption Standard*) foi desenvolvido pela IBM, com contributos da NSA, e adoptado em 1977 como padrão federal dos Estados Unidos (Stallings 2020). Durante quase duas décadas, foi o algoritmo de cifra simétrica mais usado no mundo.

O DES é uma rede de Feistel com as seguintes características:

- Bloco de **64 bits**.

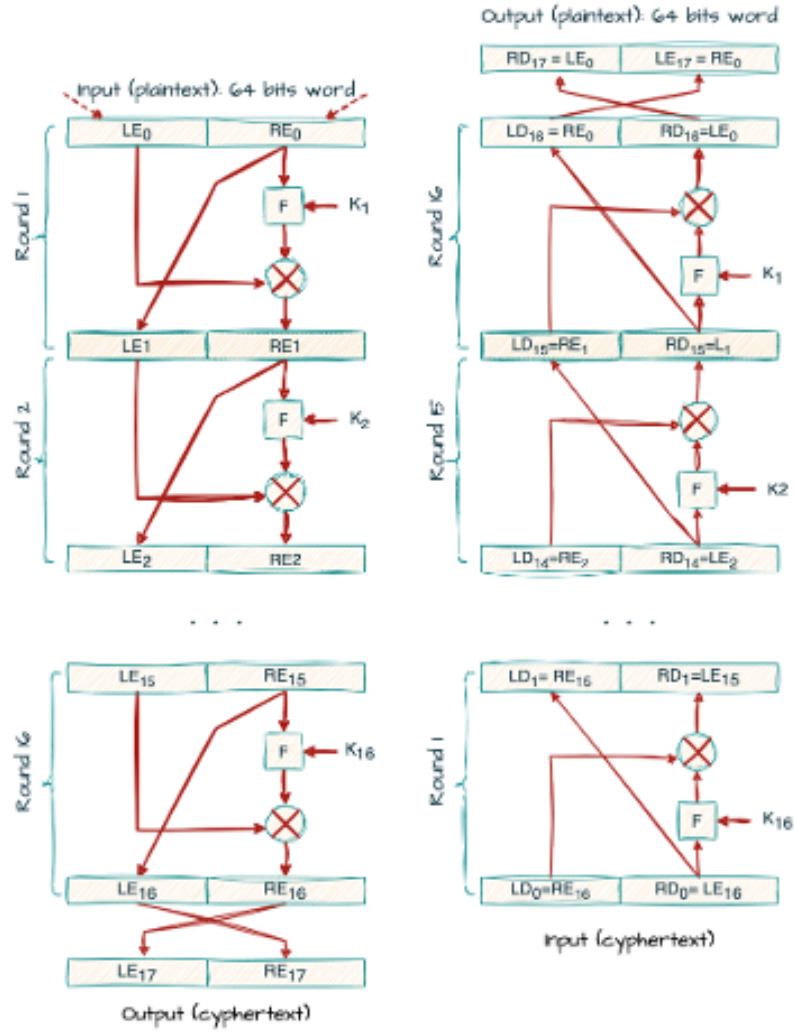


Figura 2.2: Cifragem e decifragem numa rede de Feistel de 16 rondas (o número de rondas do DES, apresentado na secção seguinte). À esquerda, a cifragem: o texto claro entra como LE_0/RE_0 e, ronda a ronda, as subchaves $K_1, K_2, \dots, K_{16}$ são aplicadas por ordem, produzindo o texto cifrado. À direita, a decifragem parte desse mesmo texto cifrado e aplica exactamente a mesma estrutura de rondas, com a mesma função F , mas com as subchaves por ordem inversa (K_{16} na primeira ronda da decifragem, K_1 na última), recuperando ao fim das 16 rondas o texto claro original.

- Chave de **56 bits** efectivos (representada como 64 bits, dos quais 8 são bits de paridade, sem contribuição para a segurança).
- **16 rondas**, cada uma com uma subchave de 48 bits derivada da chave principal por um esquema de rotações e permutações.
- Função de ronda F construída a partir de expansão de bits, XOR com a subchave, substituição não linear através de oito tabelas fixas (as **S-boxes**, a principal fonte de confusão do algoritmo) e uma permutação final.
- Uma **permutação inicial** (IP) antes da primeira ronda e a sua inversa, a **permutação final** (FP), depois da última, ambas fixas e sem qualquer contribuição para a segurança criptográfica: servem apenas para reorganizar os bits de entrada e saída, um resquício da implementação em hardware original do DES.

A estrutura geral do DES é, portanto, exactamente a rede de Feistel já descrita na secção anterior, com $n = 64$ e 16 rondas. O que é específico do DES, e não decorre da estrutura de Feistel em si, é a construção interna da função F :

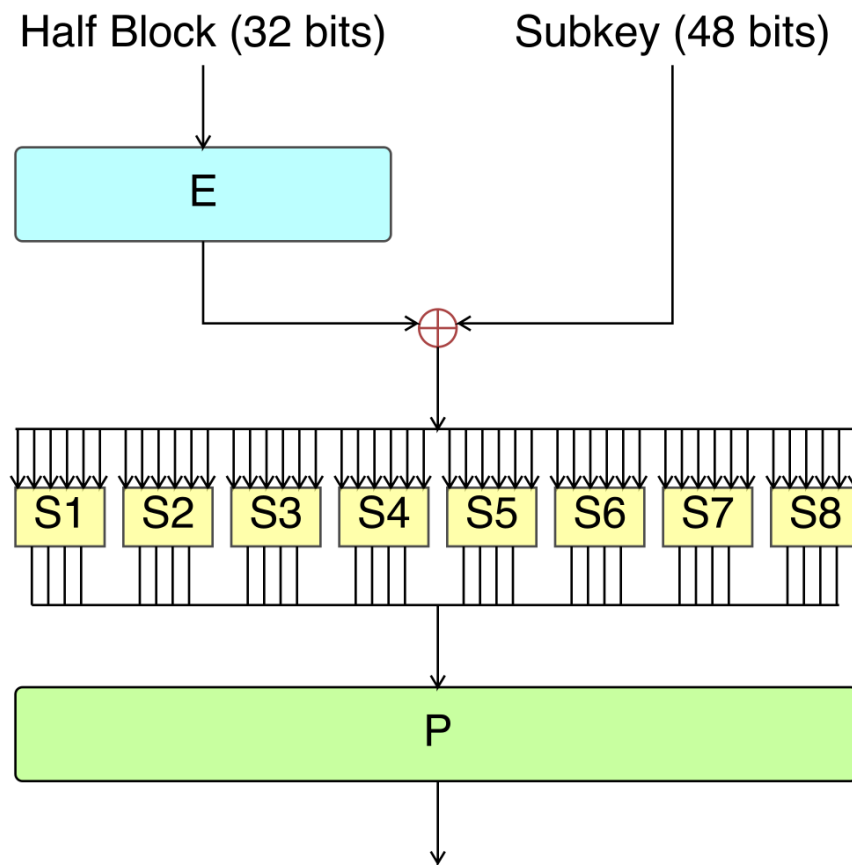


Figura 2.3: A função F do DES: a metade de 32 bits é expandida (E) para 48 bits, combinada por XOR com a subchave da ronda, dividida em oito grupos de 6 bits que atravessam as respectivas S-boxes (S1–S8, cada uma devolvendo 4 bits de saída não linear), e finalmente reorganizada por uma permutação (P). Fonte: Wikimedia Commons, domínio público.

2.3.1 A força do DES

O espaço de chaves do DES tem $2^{56} \approx 7,2 \times 10^{16}$ chaves possíveis. Este número pareceu astronomicamente seguro em 1977, mas a evolução do poder computacional tornou-o cada vez mais vulnerável a um ataque de força bruta simples, exactamente o mesmo princípio já

discutido para a cifra de César, apenas com um espaço de chaves maior. Houve ainda, desde cedo, uma segunda fonte de desconfiança: as S-boxes do DES, a componente responsável pela sua confusão, foram especificadas pela NSA sem justificação pública do critério de desenho, o que alimentou décadas de suspeitas de que pudessem esconder uma fraqueza deliberada, nunca confirmada, mas também nunca inteiramente afastada (Du 2022).

A queda do DES por força bruta ficou documentada por uma série de desafios públicos da RSA Security. Em 1997, o projecto DESCHALL quebrou uma mensagem cifrada com DES em 96 dias, usando ciclos ociosos de computadores voluntários espalhados pela Internet. Em 1998, o projecto distributed.net repetiu o feito em apenas 39 dias, e a *Electronic Frontier Foundation* construiu uma máquina dedicada, a *Deep Crack*, capaz de o fazer em cerca de 56 horas, por um custo então inferior a 250 mil dólares. Em 1999, a combinação de distributed.net com a Deep Crack reduziu o tempo a 22 horas (Du 2022; Stallings 2020). O DES foi formalmente substituído pelo 3DES em 1999, e está hoje obsoleto: não deve ser usado para proteger informação nova, apenas encontrado em sistemas legados ainda por substituir.

2.4 3DES — Triple DES

Perante a fragilidade do DES, a solução mais imediata, adoptada em 1999 enquanto se preparava um sucessor definitivo (viria a ser o AES, apresentado no capítulo seguinte), foi aplicar o próprio DES três vezes seguidas, com chaves diferentes. O **3DES** (ou *Triple DES*) cifra cada bloco segundo o esquema **EDE** (*Encrypt-Decrypt-Encrypt*):

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$

onde E e D são as operações de cifragem e decifragem do DES original. A alternância cifra-decifra-cifra, em vez de três cifragens seguidas, existe sobretudo por compatibilidade: com $K_1 = K_2 = K_3$, o 3DES reduz-se exactamente a uma única cifragem DES normal, permitindo que sistemas antigos continuem a interoperar com sistemas novos.

Consoante o número de chaves distintas usadas, o 3DES oferece:

- **Três chaves distintas** (K_1, K_2, K_3): segurança efectiva de aproximadamente 112 bits (não 168, devido a um tipo de ataque conhecido como *meet-in-the-middle*, que reduz o ganho teórico de aplicar três chaves de 56 bits).
- **Duas chaves distintas** ($K_1 = K_3$): segurança efectiva de aproximadamente 80 bits, hoje considerada insuficiente.

O 3DES resolveu o problema imediato do espaço de chaves, mas herdou as restantes limitações estruturais do DES (bloco pequeno de 64 bits, entre outras), e é cerca de três vezes mais lento do que uma única cifragem DES, uma vez que repete o algoritmo original três vezes por bloco. Por estas razões, o NIST recomenda, desde 2023, que o 3DES deixe de ser usado em aplicações novas, reservando-o apenas para manter a compatibilidade com sistemas legados (Stallings 2020). O capítulo seguinte introduz o algoritmo que o substituiu: o AES.

2.5 Síntese do Capítulo

Este capítulo introduziu a estrutura fundamental das cifras de bloco modernas:

1. Uma **cifra de bloco** é uma permutação de blocos de bits de tamanho fixo, parametrizada pela chave, generalizando a ideia de permutação já vista na cifra monoalfabética, mas num espaço muito maior
2. A **cifra ideal**, que realizaria essa permutação de forma completamente arbitrária, exigiria uma chave de $n \times 2^n$ bits, impraticável para qualquer bloco realista; as cifras de bloco reais aproximam-se do seu comportamento por outra via
3. **Confusão** (relação complexa chave-texto cifrado) e **difusão** (espalhamento da influência de cada bit), propostas por Shannon, são as duas propriedades que as cifras de bloco modernas combinam ao longo de várias rondas para conseguir essa aproximação
4. A **rede de Feistel** organiza estas rondas de forma a que a decifragem use exactamente a mesma estrutura da cifragem, com as subchaves por ordem inversa, mesmo que a função de ronda F não seja, em si, invertível
5. O **DES**, padrão dominante entre 1977 e 1999, tem bloco de 64 bits, chave efectiva de 56 bits e 16 rondas de Feistel; o seu espaço de chaves reduzido tornou-o vulnerável a força bruta (de 96 dias, em 1997, a 22 horas, em 1999), e está hoje obsoleto
6. O **3DES**, adoptado em 1999, aplica o DES três vezes (esquema EDE), estendendo a segurança efectiva até cerca de 112 bits, ao custo de ser cerca de três vezes mais lento; está também a caminho da descontinuação

2.6 Para Pensar

Na rede de Feistel, a função F não precisa de ser invertível para que todo o sistema seja reversível. Que vantagem prática traz isto a quem desenha o algoritmo, comparando com uma cifra em que a função central tivesse de ser, ela própria, invertível?

Referências

- AlFardan, Nadhem J., Daniel J. Bernstein, Kenneth G. Paterson, Bertram Poettering, e Jacob C. N. Schuldt. 2013. «On the Security of RC4 in TLS». Em *22nd USENIX Security Symposium*.
- Aumasson, Jean-Philippe. 2017. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press.
- Böck, Hanno, Aaron Zauner, Sean Devlin, Juraj Somorovsky, e Philipp Jovanovic. 2016. «Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS». Em *USENIX Workshop on Offensive Technologies (WOOT)*.
- Du, Wenliang. 2022. *Computer Security: A Hands-on Approach*. 3.^a ed. CreateSpace / Self-published.
- Dworkin, Morris. 2001. «Recommendation for Block Cipher Modes of Operation: Methods and Techniques». SP 800-38A. NIST.
- . 2007. «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC». SP 800-38D. NIST.
- Fluhrer, Scott, Itsik Mantin, e Adi Shamir. 2001. «Weaknesses in the Key Scheduling Algorithm of RC4». Em *Selected Areas in Cryptography (SAC 2001)*.
- Kerckhoffs, Auguste. 1883. «La cryptographie militaire». *Journal des Sciences Militaires* 9: 5–38.
- Martin, Keith. 2017. *Everyday Cryptography*. 2.^a ed. Oxford University Press.
- National Institute of Standards and Technology. 2001. «Advanced Encryption Standard (AES)». FIPS PUB 197. NIST.

Referências

- Nir, Yoav, e Adam Langley. 2018. «ChaCha20 and Poly1305 for IETF Protocols». RFC 8439. IETF.
- Popov, Andrei. 2015. «Prohibiting RC4 Cipher Suites». RFC 7465. IETF.
- Shannon, Claude E. 1949. «Communication Theory of Secrecy Systems». *Bell System Technical Journal* 28 (4): 656–715.
- Stallings, William. 2020. *Cryptography and Network Security: Principles and Practice*. 8.^a ed. Pearson.

3

AES — Estágios e Modos de Operação

Introdução

O capítulo anterior terminou com uma conclusão incômoda: o DES, durante décadas o pilar da cifra simétrica, está obsoleto, e o 3DES, a solução de recurso, é lento e continua limitado pelo bloco de 64 bits herdado do original. Era necessário um sucessor desenhado de raiz, não remendado. Este capítulo apresenta esse sucessor: o **AES** (*Advanced Encryption Standard*), o algoritmo de cifra simétrica hoje universalmente adotado, de aplicações bancárias a comunicações protegidas por TLS. A segunda parte do capítulo trata de uma questão distinta, mas igualmente prática: uma cifra de bloco só cifra um bloco de cada vez, então como se cifra uma mensagem real, quase sempre maior do que isso?

3.1 Do DES ao AES: um Novo Paradigma

Em 1997, o *National Institute of Standards and Technology* (NIST) dos Estados Unidos lançou um concurso público e aberto para escolher o sucessor do DES, ao contrário do processo fechado que tinha originado o próprio DES duas décadas antes. Qualquer equipa, de qualquer país, podia submeter um algoritmo, que seria depois escrutinado publicamente por toda a comunidade criptográfica mundial.

Quinze algoritmos foram submetidos. Cinco chegaram à fase final: MARS, RC6, Rijndael, Serpent e Twofish. Em Outubro de 2000, o NIST anunciou o vencedor: **Rijndael**, desenhado pelos criptógrafos belgas Joan Daemen e Vincent Rijmen. O algoritmo foi formalizado como norma federal em 2001, sob o nome AES (National Institute of Standards and Technology 2001).

A diferença estrutural mais importante em relação ao DES não é o tamanho da chave ou do bloco, é a arquitectura interna. O DES, como se viu, é uma rede de Feistel: em cada ronda, apenas metade do bloco atravessa a função não linear F , enquanto a outra metade passa praticamente incólume, apenas combinada por XOR. O AES abandona esta estrutura. É uma **rede de substituição-permutação** (*substitution-permutation network*, SPN): em cada ronda, o **bloco inteiro** é transformado, através de uma sequência de quatro operações distintas aplicadas a todos os seus bytes.

3.2 Estrutura Geral do AES

Ao contrário do DES, em que o tamanho do bloco (64 bits) e o da chave (56 bits efectivos) estão fixados pela norma, o AES separa estas duas dimensões:

- **Bloco:** sempre **128 bits** (16 bytes), independentemente do tamanho da chave.
- **Chave:** **128, 192 ou 256 bits**, dando origem a três variantes — AES-128, AES-192 e AES-256.
- **Número de rondas:** depende do tamanho da chave — **10 rondas** para AES-128, **12** para AES-192, **14** para AES-256. Chaves maiores implicam mais rondas, porque cada ronda adicional dificulta ainda mais qualquer tentativa de análise estatística ou algébrica do algoritmo.

Internamente, o bloco de 128 bits organiza-se numa matriz 4×4 de bytes, a que se chama **estado** (*state*), preenchida coluna a coluna: os primeiros 4 bytes do bloco ocupam a primeira coluna, os 4 seguintes a segunda, e assim sucessivamente. Todas as transformações de ronda operam sobre esta matriz.

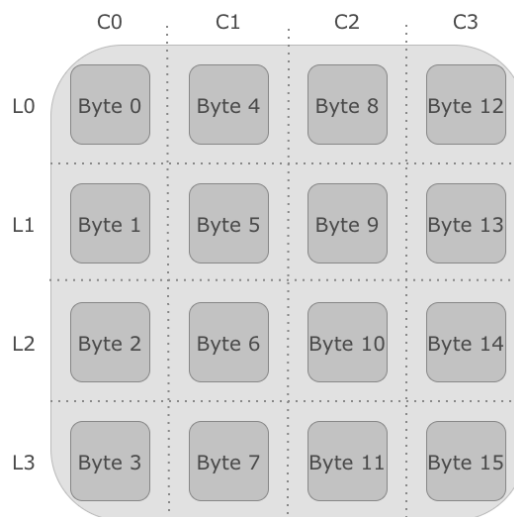


Figura 3.1: O estado do AES: o bloco de 128 bits (16 bytes) organiza-se numa matriz 4×4 , preenchida coluna a coluna (byte 0 a byte 3 na primeira coluna, byte 4 a byte 7 na segunda, e assim sucessivamente).

O AES puro é **determinístico**: o mesmo bloco de texto claro, com a mesma chave, produz sempre o mesmo bloco de texto cifrado. Esta previsibilidade é inaceitável na prática — é precisamente o problema que, como se verá adiante, obriga a recorrer a um modo de operação, que introduz aleatoriedade através de um vector de inicialização ou *nonce*.

3.3 As Quatro Transformações de Ronda

Cada ronda do AES (excepto a última) aplica, por esta ordem, quatro transformações à matriz de estado. Antes da primeira ronda, aplica-se ainda um **AddRoundKey** inicial, e a última ronda omite o **MixColumns**. O objectivo destas quatro operações não é apenas ocultar o texto original, mas fazê-lo de forma que uma alteração mínima, um único bit, na mensagem ou na chave, produza alterações significativas e imprevisíveis no texto cifrado — o chamado **efeito avalanche**.

3.3.1 SubBytes

SubBytes é a única operação não linear do AES, e é ela que fornece a **confusão**: cada byte do estado é substituído pelo byte correspondente numa tabela fixa de substituição, a **S-box**, construída a partir do inverso multiplicativo no corpo finito $GF(2^8)$ seguido de uma transformação afim. Ao contrário do DES, que usa oito S-boxes diferentes (uma por grupo de 6 bits), o AES usa uma **única S-box**, reaplicada byte a byte a toda a matriz de estado.

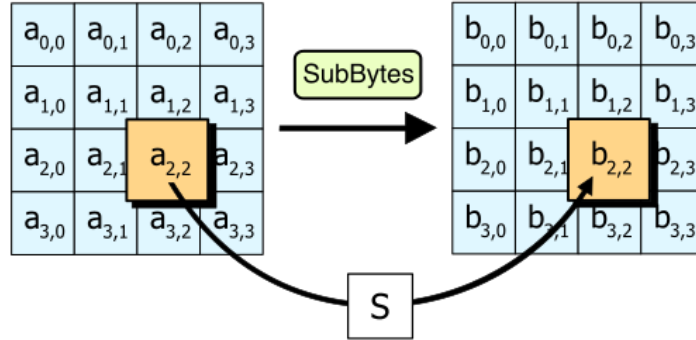


Figura 3.2: No passo SubBytes, cada byte do estado é substituído pelo byte correspondente numa S-box fixa. Fonte: Wikimedia Commons, domínio público.

3.3.2 ShiftRows

ShiftRows desloca ciclicamente cada linha da matriz de estado para a esquerda: a linha 0 não se altera, a linha 1 desloca-se 1 posição, a linha 2 desloca-se 2 posições, a linha 3 desloca-se 3 posições. Esta operação é simples, mas essencial: garante que os bytes de uma coluna deixam de pertencer todos à mesma coluna, espalhando a influência de cada byte por várias colunas nas rondas seguintes.

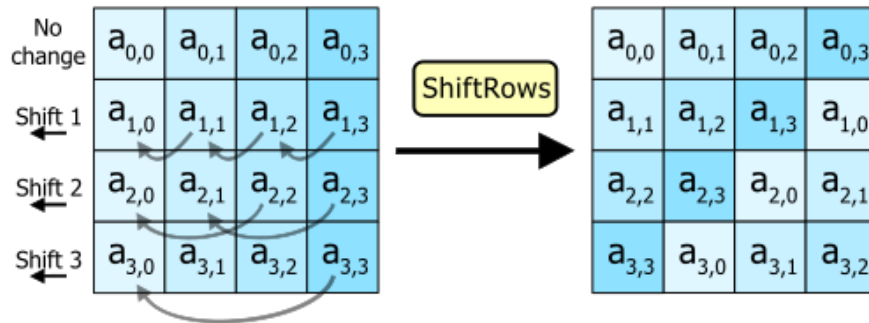


Figura 3.3: No passo ShiftRows, cada linha da matriz de estado é deslocada ciclicamente para a esquerda, um número de posições diferente por linha. Fonte: Wikimedia Commons, domínio público.

3.3.3 MixColumns

MixColumns opera sobre cada coluna da matriz de estado independentemente, tratando-a como um polinómio de grau 3 sobre $GF(2^8)$ e multiplicando-a por um polinómio fixo $c(x)$, módulo $x^4 + 1$. O efeito prático é que cada byte de saída de uma coluna passa a depender dos quatro bytes de entrada dessa mesma coluna. Combinado com o **ShiftRows** anterior, garante

que, ao fim de poucas rondas, cada byte de saída depende de todos os bytes de entrada do bloco: é a **difusão** do AES. Esta transformação omite-se na última ronda.

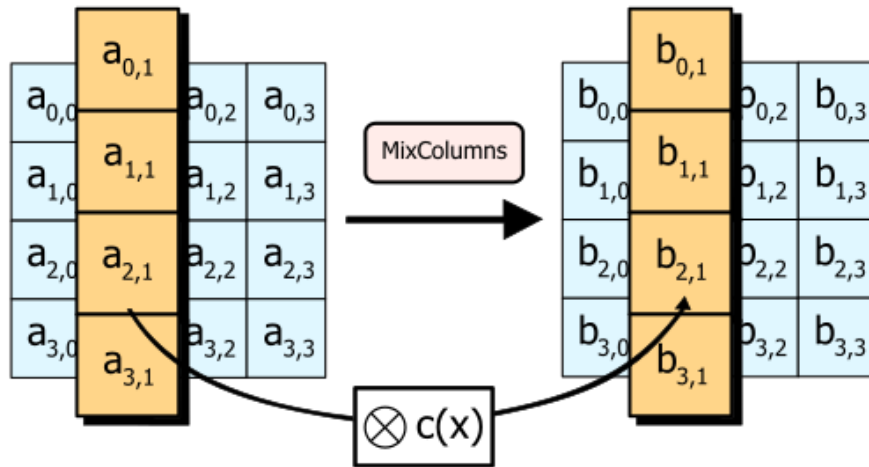


Figura 3.4: No passo MixColumns, cada coluna da matriz de estado é multiplicada por um polinômio fixo $c(x)$, misturando os seus quatro bytes. Fonte: Wikimedia Commons, domínio público.

3.3.4 AddRoundKey

AddRoundKey é a única transformação que envolve a chave: cada byte do estado combina-se por XOR com o byte correspondente da subchave dessa ronda, derivada da chave principal pelo **esquema de expansão de chave** (*key schedule*). É também a única transformação que, isoladamente, seria trivial de inverter sem conhecer a chave, o que confirma que a segurança do AES depende da combinação das quatro transformações, e não de nenhuma isoladamente.

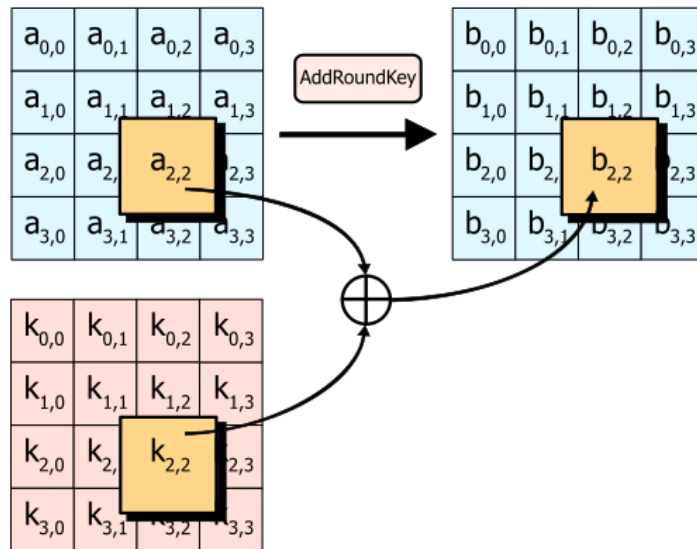


Figura 3.5: No passo AddRoundKey, cada byte do estado combina-se por XOR com o byte correspondente da subchave da ronda. Fonte: Wikimedia Commons, domínio público.

3.4 A Ronda Completa e a Expansão de Chave

Uma ronda intermédia do AES aplica, por esta ordem, `SubBytes`, `ShiftRows`, `MixColumns` e `AddRoundKey`. A última ronda omite o `MixColumns`, e há ainda um `AddRoundKey` inicial, antes de qualquer ronda, usando a própria chave original sem expansão.

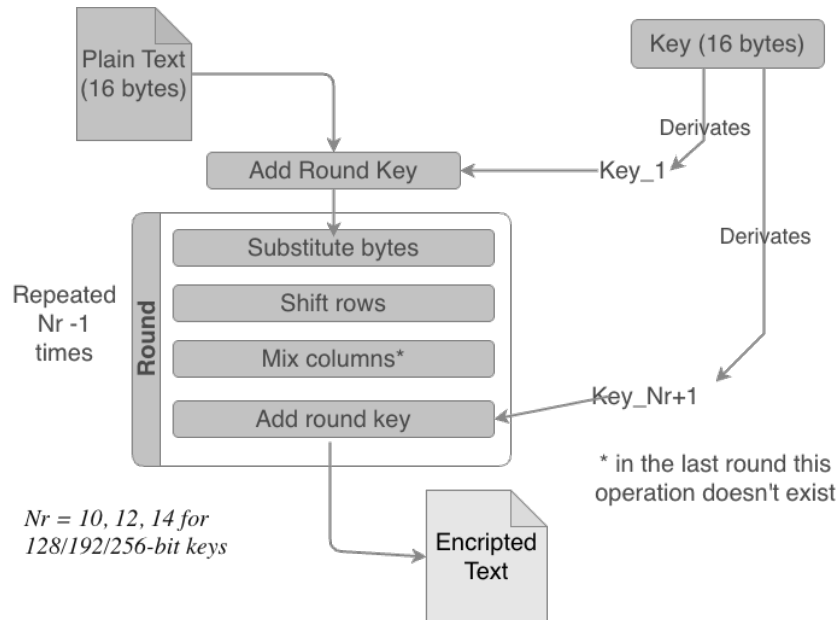


Figura 3.6: A estrutura completa do AES: um `AddRoundKey` inicial, seguido de $N_r - 1$ rondas intermédias (`SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey`), e uma ronda final que omite o `MixColumns`.

Isto significa que um AES de N_r rondas precisa de $N_r + 1$ subchaves de 128 bits: uma para o `AddRoundKey` inicial, mais uma por ronda. Estas subchaves não são fragmentos independentes da chave original, mas são geradas por um **esquema de expansão de chave**: a chave original é processada palavra a palavra (grupos de 4 bytes), envolvendo rotações, substituições pela mesma S-box do `SubBytes` e combinação com constantes de ronda fixas, produzindo a sequência completa de subchaves necessárias. O detalhe exacto deste esquema foge ao âmbito deste capítulo; o essencial é reter que todas as subchaves derivam de uma única chave secreta, e que a sua geração é determinística e pública, tal como toda a restante estrutura do algoritmo — a segurança do AES reside inteiramente na chave, nunca no segredo do desenho, tal como o princípio de Kerckhoffs exige.

3.5 Segurança do AES

Desde a sua normalização em 2001, o AES foi sujeito a um escrutínio criptanalítico intenso e contínuo, sem que se tenha encontrado qualquer ataque prático capaz de o quebrar mais rapidamente do que a força bruta. O espaço de chaves mínimo, 2^{128} , torna a força bruta pura astronomicamente inviável, muito além do que já era o caso para o DES.

Na prática, as vulnerabilidades reais associadas ao AES não são matemáticas, mas de implementação: **ataques de canal lateral** (*side-channel*), que exploram informação indirecta como o tempo de execução ou o consumo energético de um dispositivo a cifrar, em vez de atacar o algoritmo em si. É por esta razão que os processadores modernos incluem instruções

dedicadas ao AES (como o conjunto AES-NI), que executam as transformações em tempo constante, precisamente para eliminar esta classe de ataques.

Vale ainda a pena antecipar uma questão sobre computação quântica: o algoritmo de Grover permite, em teoria, reduzir a segurança efectiva de uma pesquisa por força bruta para aproximadamente metade do número de bits da chave, o que tornaria o AES-128 equivalente a cerca de 64 bits de segurança clássica, um valor hoje considerado insuficiente. É por isso que o AES-256, apesar de mais lento, é a opção recomendada para dados que devam permanecer protegidos a muito longo prazo. O capítulo dedicado à criptografia pós-quântica, mais adiante, retoma esta questão em detalhe.

3.6 Modos de Operação

Uma cifra de bloco, seja o AES ou o DES, cifra exactamente um bloco de cada vez. Uma mensagem real quase nunca tem exactamente o tamanho de um bloco (128 bits, no caso do AES) — normalmente é maior, por vezes muito maior. Um **modo de operação** define como aplicar repetidamente a mesma cifra de bloco a uma sequência de blocos, de forma a cifrar uma mensagem de qualquer tamanho.

3.6.1 ECB — Electronic Codebook

O modo mais simples divide a mensagem em blocos e cifra cada um independentemente, com a mesma chave: $C_i = E_K(P_i)$. É simples e paralelizável, mas tem um defeito grave: **blocos de texto claro idênticos produzem sempre blocos de texto cifrado idênticos**. Isto é exactamente o mesmo problema de fundo já visto na cifra monoalfabética, só que ao nível de blocos de 128 bits em vez de letras individuais: qualquer padrão ou repetição estrutural da mensagem original permanece visível no texto cifrado, mesmo sem se decifrar nada.

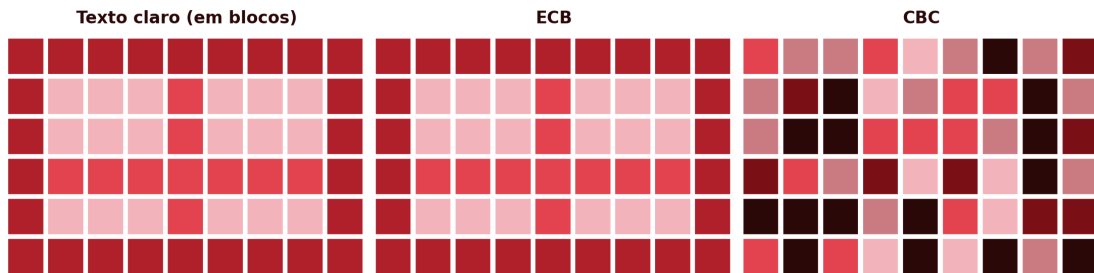


Figura 3.7: Um texto claro organizado em blocos, com um padrão estrutural óbvio (moldura, cruz), cifrado em modo ECB e em modo CBC. Sob ECB, blocos de texto claro idênticos geram blocos de texto cifrado idênticos: o padrão original mantém-se perfeitamente visível, apenas com cores (valores) diferentes. Sob CBC, o mesmo padrão desaparece por completo.

3.6.2 CBC — Cipher Block Chaining

O CBC resolve este problema encadeando os blocos: antes de cifrar cada bloco de texto claro, combina-se por XOR com o bloco de texto cifrado anterior.

$$C_i = E_K(P_i \oplus C_{i-1})$$

Como não existe um “bloco anterior” para o primeiro bloco da mensagem, usa-se um **vector de inicialização** (*initialization vector*, IV): um bloco aleatório, gerado de novo em cada mensagem, que substitui C_0 . O IV não precisa de ser secreto, apenas único e imprevisível, e é normalmente transmitido em claro junto com o texto cifrado.

Como cada bloco cifrado depende de todos os blocos de texto claro anteriores (através da cadeia de XORs), blocos de texto claro idênticos deixam de produzir blocos de texto cifrado idênticos, como se vê na Figura 3.7: o padrão original desaparece completamente. A decifragem inverte exactamente este processo:

$$P_i = D_K(C_i) \oplus C_{i-1}$$

Note-se uma assimetria entre as duas direcções: a **decifragem** em CBC é paralelizável, porque cada bloco decifrado só depende do bloco de texto cifrado anterior, já disponível na totalidade; a **cifragem**, pelo contrário, não é, porque cada bloco só pode ser cifrado depois de o anterior estar pronto, uma vez que é o seu resultado que entra na cadeia seguinte.

3.6.3 CFB — Cipher Feedback Mode

O **CFB** consegue um efeito semelhante ao CBC, eliminar o padrão do ECB, mas por um caminho diferente: em vez de cifrar o texto claro directamente, cifra-se o bloco de texto cifrado anterior, e o resultado combina-se por XOR com o texto claro para produzir o bloco seguinte.

$$C_i = P_i \oplus E_K(C_{i-1})$$

com $C_0 = IV$. Tal como no CBC, o IV não pode ser reutilizado, mas pode ser público. O CFB comporta-se, na prática, como uma **cifra de pseudo-fluxo**: em vez de cifrar blocos completos, pode operar sobre segmentos mais pequenos (por exemplo, byte a byte), o que o torna útil para fluxos de dados de comprimento variável. Tal como no CBC, a cifragem não é paralelizável, mas a decifragem é.

3.6.4 CTR — Counter Mode

O **CTR** segue uma abordagem diferente: em vez de cifrar o texto claro directamente, cifra-se um contador (combinado com um valor único por mensagem, o *nonce*), e o resultado combina-se por XOR com o texto claro:

$$C_i = P_i \oplus E_K(\text{nonce} \parallel i)$$

Isto transforma, na prática, uma cifra de bloco numa cifra de fluxo, tema do próximo capítulo: cada bloco de saída de E_K funciona como uma sequência de bits pseudo-aleatória, gerada independentemente do texto claro. A vantagem prática mais relevante é que, ao contrário do CBC e do CFB, tanto a cifragem como a decifragem em CTR são inteiramente **paralelizáveis**: o bloco i não depende do bloco $i - 1$, apenas do próprio contador, o que se traduz em desempenho significativamente superior em hardware moderno com múltiplos núcleos.

Este ganho de desempenho tem duas contrapartidas importantes. Primeiro, **reutilizar o mesmo nonce é catastrófico**: tal como numa cifra de fluxo genuína (capítulo seguinte), reutilizá-lo produz o mesmo fluxo pseudo-aleatório para duas mensagens diferentes, expondo-as

ao ataque do *two-time pad*. Segundo, o CTR é **maleável**: como a cifração actua sobre o contador, não sobre o texto claro, a propriedade de difusão do AES nunca chega a actuar sobre a relação entre texto cifrado e texto claro. Um atacante que altere um bit do texto cifrado altera exactamente o bit correspondente do texto claro decifrado, de forma previsível, sem que nada no CTR o detecte.

3.6.5 GCM — Galois/Counter Mode

Nenhum dos modos apresentados até aqui, ECB, CBC, CFB ou CTR, protege a **integridade** da mensagem: um atacante que intercepte o texto cifrado pode alterá-lo (o CTR, como se viu, é particularmente vulnerável a isto), e nenhum destes modos detecta a alteração — só garantem que o atacante não consegue ler o conteúdo original, não que não o consiga adulterar.

O **GCM** (*Galois/Counter Mode*) resolve isto: estende o modo CTR (do qual herda a confidencialidade e a total paralelização) com um mecanismo de autenticação baseado numa função de hash sobre um corpo finito, o **GHASH**. O resultado é uma **cifra autenticada** (*AEAD, Authenticated Encryption with Associated Data*): produz, além do texto cifrado, uma marca de autenticação (*tag*) de 128 bits, calculada não só sobre o texto cifrado mas também sobre eventuais **dados associados** não cifrados (*additional authenticated data, AAD*), como cabeçalhos de rede ou metadados que precisam de ser autenticados mas não escondidos. Qualquer alteração, ao texto cifrado ou ao AAD, invalida a marca.

Tal como no CTR, a reutilização do *nonce* (recomendado a 96 bits) em GCM é catastrófica, e já foi explorada em ataques práticos de falsificação contra implementações reais de TLS (Böck et al. 2016). Por combinar confidencialidade, integridade e paralelização total, o GCM é hoje o modo **recomendado** para a generalidade das aplicações novas, usado, entre outros, no TLS 1.3, no SSH, no IPsec e no QUIC.

3.6.6 Comparação dos modos de operação

Tabela 3.1: Comparação dos modos de operação apresentados neste capítulo (Dworkin 2001, 2007)

Modo	IV / nonce	Paralelizável (cifra / decifra)	Autenticação nativa	Uso recomendado
ECB	Nenhum	Sim / Sim	Não	Nunca (revela padrões)
CBC	IV aleatório	Não / Sim	Não	Uso legado; substituir por AEAD
CFB	IV aleatório	Não / Sim	Não	Fluxos de dados de comprimento variável
CTR	Nonce único	Sim / Sim	Não	Alto débito, sem necessidade de autenticação
GCM	Nonce único (96 bits)	Sim / Sim	Sim (GHASH, 128 bits)	Recomendado (TLS 1.3, SSH, IPsec, QUIC)

3.7 Síntese do Capítulo

Este capítulo apresentou o algoritmo que hoje ocupa o lugar do DES, e a forma como se aplica a mensagens de qualquer tamanho:

1. O **AES** resultou de um concurso público do NIST, vencido pelo algoritmo **Rijndael**, e normalizado em 2001
2. Ao contrário do DES, o AES **não é uma rede de Feistel**: é uma **rede de substituição-permutação** (SPN), em que cada ronda transforma o bloco inteiro
3. O bloco é sempre de **128 bits**; a chave pode ser de **128, 192 ou 256 bits**, com **10, 12 ou 14 rondas**, respectivamente
4. Cada ronda aplica quatro transformações: **SubBytes** (confusão, via S-box não linear), **ShiftRows** e **MixColumns** (difusão, entre e dentro de colunas), e **AddRoundKey** (mistura com a subchave)
5. Não existe, até hoje, nenhum ataque criptanalítico prático contra o AES; as vulnerabilidades reais são de implementação (ataques de canal lateral), não do algoritmo em si
6. Um **modo de operação** define como aplicar uma cifra de bloco a mensagens maiores do que um bloco: o **ECB** é simples mas revela padrões estruturais; o **CBC** e o **CFB** encadeiam os blocos e eliminam-os, à custa da cifragem não ser paralelizável; o **CTR** transforma a cifra de bloco numa cifra de fluxo, totalmente paralelizável, mas maleável
7. Nenhum destes modos protege a integridade da mensagem; o **GCM** resolve isto combinando o CTR com autenticação (GHASH), numa construção **AEAD** que é hoje o modo recomendado para a generalidade das aplicações novas

3.8 Para Pensar

O DES combina confusão e difusão através de uma única função F , aplicada apenas a metade do bloco por ronda. O AES separa estas duas propriedades em transformações distintas (**SubBytes** para a confusão, **ShiftRows**/**MixColumns** para a difusão), aplicadas ao bloco inteiro. Que vantagens pode trazer esta separação explícita, tanto para a análise de segurança do algoritmo como para o seu desempenho em hardware?

Referências

- AlFardan, Nadhem J., Daniel J. Bernstein, Kenneth G. Paterson, Bertram Poettering, e Jacob C. N. Schuldt. 2013. «On the Security of RC4 in TLS». Em *22nd USENIX Security Symposium*.
- Aumasson, Jean-Philippe. 2017. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press.
- Böck, Hanno, Aaron Zauner, Sean Devlin, Juraj Somorovsky, e Philipp Jovanovic. 2016. «Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS». Em *USENIX Workshop on Offensive Technologies (WOOT)*.
- Du, Wenliang. 2022. *Computer Security: A Hands-on Approach*. 3.^a ed. CreateSpace / Self-published.
- Dworkin, Morris. 2001. «Recommendation for Block Cipher Modes of Operation: Methods and Techniques». SP 800-38A. NIST.
- . 2007. «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC». SP 800-38D. NIST.

Referências

- Fluhrer, Scott, Itsik Mantin, e Adi Shamir. 2001. «Weaknesses in the Key Scheduling Algorithm of RC4». Em *Selected Areas in Cryptography (SAC 2001)*.
- Kerckhoffs, Auguste. 1883. «La cryptographie militaire». *Journal des Sciences Militaires* 9: 5–38.
- Martin, Keith. 2017. *Everyday Cryptography*. 2.^a ed. Oxford University Press.
- National Institute of Standards and Technology. 2001. «Advanced Encryption Standard (AES)». FIPS PUB 197. NIST.
- Nir, Yoav, e Adam Langley. 2018. «ChaCha20 and Poly1305 for IETF Protocols». RFC 8439. IETF.
- Popov, Andrei. 2015. «Prohibiting RC4 Cipher Suites». RFC 7465. IETF.
- Shannon, Claude E. 1949. «Communication Theory of Secrecy Systems». *Bell System Technical Journal* 28 (4): 656–715.
- Stallings, William. 2020. *Cryptography and Network Security: Principles and Practice*. 8.^a ed. Pearson.

4

Cifras de Fluxo — RC4, Salsa20, ChaCha20 e Padding

Introdução

Os capítulos anteriores trataram sempre de **cifras de bloco**: DES e AES processam a mensagem em fragmentos de tamanho fixo, 64 ou 128 bits de cada vez, e precisaram de um modo de operação para lidar com mensagens de qualquer comprimento. Este capítulo apresenta a outra grande família de cifras simétricas, as **cifras de fluxo**, que dispensam essa mecânica: geram uma sequência pseudo-aleatória de bits, byte a byte, tão longa quanto a mensagem, e combinam-na com o texto claro por XOR, sem qualquer noção de bloco. As cifras de fluxo têm ganho renovada relevância nos últimos anos, precisamente por serem rápidas a processar fluxos contínuos de dados e por funcionarem bem em dispositivos com poucos recursos, como sensores sem fios ou etiquetas RFID em sistemas *Internet of Things* (Martin 2017). Há ainda uma segunda razão, menos técnica: o descrédito generalizado das cifras de bloco em modo CBC, precisamente por causa dos ataques de oráculo de *padding* já referidos no capítulo anterior, empurrou muitos sistemas modernos na direcção das cifras de fluxo (e das construções AEAD baseadas nelas) (Aumasson 2017).

4.1 Cifras de Fluxo: o Conceito Geral

O modo CTR, apresentado no capítulo anterior, já antecipava esta ideia: usar uma cifra de bloco para gerar uma sequência de valores pseudo-aleatórios, e combiná-la com o texto claro por XOR, tal como uma cifra de fluxo genuína. Uma **cifra de fluxo** pode ser vista como um caso particular de **gerador determinístico de bits aleatórios** (*deterministic random bit generator*, DRBG): a diferença é que um DRBG genérico só recebe um valor de entrada (uma semente), enquanto uma cifra de fluxo recebe sempre dois, a chave K (secreta, tipicamente de 128 ou 256 bits) e o *nonce* N (não precisa de ser secreto, mas tem de ser único para cada chave, tipicamente entre 64 e 128 bits) (Aumasson 2017). A partir destes dois valores, um **gerador de fluxo de chave** (*keystream generator*) produz uma sequência de bits $k_1, k_2, k_3, \dots$, o **fluxo de chave** (*keystream*), tão longa quanto necessário, e a cifragem é uma simples combinação por XOR, bit a bit ou byte a byte:

$$C_i = P_i \oplus k_i$$

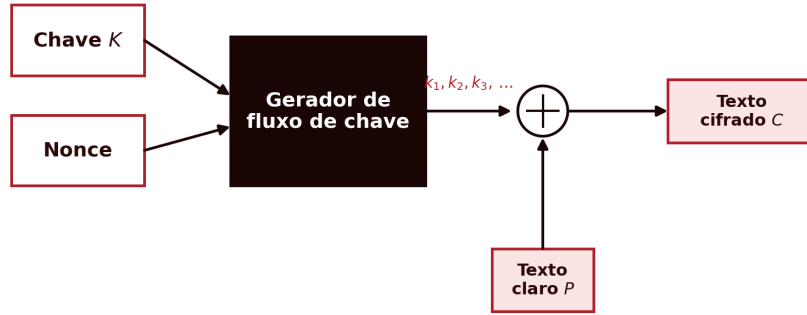


Figura 4.1: Uma cifra de fluxo genérica: a chave e o nonce alimentam um gerador pseudo-aleatório que produz o fluxo de chave $k_1, k_2, k_3, \dots$, combinado por XOR com o texto claro, byte a byte, produzindo o texto cifrado.

Esta construção devia parecer familiar: é exactamente a estrutura do **One-Time Pad**, já referido no capítulo 1 e retomado no capítulo 2 como o único sistema com segredo perfeito demonstrado por Shannon. Uma cifra de fluxo pode, por isso, ser vista como o **equivalente pseudo-aleatório de um One-Time Pad** (Martin 2017; Aumasson 2017): a diferença, e é uma diferença crucial, é que o One-Time Pad exige uma chave genuinamente aleatória, tão longa quanto a mensagem, e nunca reutilizada, enquanto uma cifra de fluxo troca essa exigência impraticável por uma chave curta e reutilizável, à custa de perder o segredo perfeito: a segurança passa a depender inteiramente da **imprevisibilidade computacional** do gerador, isto é, de ser inviável distinguir o seu fluxo de saída de uma sequência genuinamente aleatória sem conhecer a chave.

Esta troca tem uma consequência que não perdoa erros: o par (chave, *nonce*) nunca pode ser reutilizado. Note-se que isto é mais permissivo do que possa parecer à primeira vista: cifrar uma mensagem com (K_1, N_1) e outra com o mesmo K_1 mas um N_1 diferente é seguro, tal como cifrar com um K_2 diferente e o mesmo N_1 ; o que nunca pode acontecer é voltar a cifrar com exactamente o mesmo par (K_1, N_1) já usado antes (Aumasson 2017). Se isso acontecer, o gerador produz exactamente o mesmo fluxo de chave para duas mensagens diferentes. Um atacante que intercepte ambos os textos cifrados pode combiná-los por XOR entre si, e o fluxo de chave cancela-se completamente:

$$C_1 \oplus C_2 = (P_1 \oplus k) \oplus (P_2 \oplus k) = P_1 \oplus P_2$$

O resultado, o XOR dos dois textos claros, já não depende da chave, e é frequentemente suficiente para recuperar ambas as mensagens através de análise estatística da língua, tal como se fez com o método de Kasiski no capítulo 1. Este é o chamado **ataque do two-time pad**, e foi exactamente o tipo de falha, na reutilização de valores de inicialização, que tornou o protocolo WEP (a primeira norma de segurança Wi-Fi) praticamente inútil.

4.2 RC4

O **RC4**, desenhado por Ron Rivest na RSA Security em 1987, foi mantido como segredo comercial até ser divulgado, sem autorização, em 1994. Aceita chaves de comprimento variável, entre 40 e 2048 bits, e mantém um estado interno de 256 bytes (um *array* S , com dois ponteiros auxiliares i e j). É orientado a byte (não a bit), e organiza-se em duas fases:

- **KSA** (*Key Scheduling Algorithm*): a partir da chave, constrói e permuta o *array* S de 256 valores (inicialmente $S[i] = i$), misturando-o repetidamente com os bytes da chave.
- **PRGA** (*Pseudo-Random Generation Algorithm*): a cada iteração, actualiza os dois ponteiros i e j , troca $S[i]$ com $S[j]$, e produz um byte do fluxo de chave.

$S[]$ — 256 bytes, permutados pela KSA

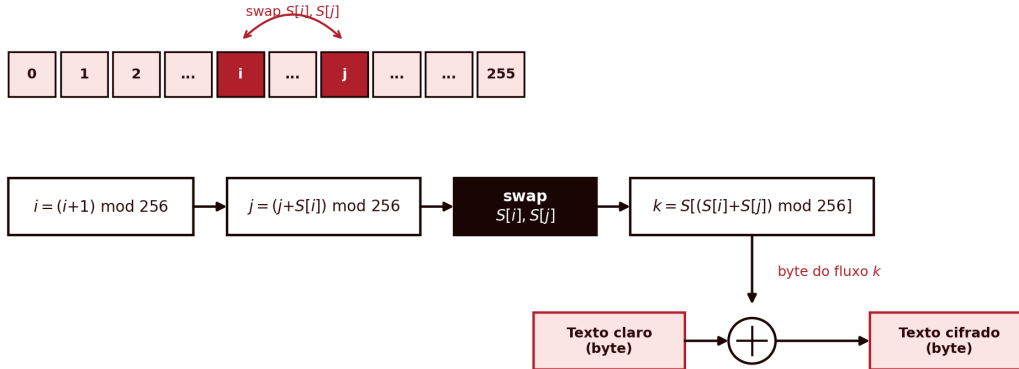


Figura 4.2: O mecanismo interno do RC4: a KSA permuta o array S de 256 bytes a partir da chave; a PRGA, a cada iteração, actualiza os índices i e j , troca $S[i]$ com $S[j]$, e extrai um byte do fluxo de chave, combinado por XOR com o texto claro.

Um detalhe do desenho do RC4 vale a pena assinalar: **não existe nenhum mecanismo de *nonce* incorporado no algoritmo** — “como não acontece numa cifra de fluxo decente”, nas palavras do criptógrafo Jean-Philippe Aumasson (Aumasson 2017). Qualquer *nonce* tem de ser adicionado externamente, por quem implementa o protocolo, e foi precisamente aqui que o WEP falhou: prefixava a chave com um *nonce* curto e previsível antes de cada KSA, tornando o sistema vulnerável ao **ataque de Fluhrer, Mantin e Shamir** (2001) (Fluhrer, Mantin, e Shamir 2001), que explora o facto de os primeiros bytes do fluxo de chave do RC4 não serem estatisticamente uniformes. O caso mais conhecido: o **segundo byte** do fluxo de chave vale zero com uma probabilidade de $1/128$, o dobro do que seria de esperar ($1/256$) num byte verdadeiramente aleatório (Aumasson 2017) — um desvio pequeno, mas suficiente para, com milhões de mensagens interceptadas, distinguir estatisticamente o RC4 de uma fonte aleatória e, em certas condições, recuperar bytes do texto claro sem nunca descobrir a chave. A mitigação proposta, descartar os primeiros 3072 bytes do fluxo antes de o usar (RC4-drop3072), reduz mas não elimina o problema. Um segundo ataque prático, contra o próprio TLS, foi publicado em 2013 (AlFardan et al. 2013). Confrontado com estas fragilidades acumuladas, o RC4 foi formalmente banido do TLS pela *Internet Engineering Task Force* em 2015 (Popov 2015), e não deve ser usado em nenhum sistema novo.

4.3 Duas Arquitecturas: Stateful vs. por Contador

Antes de avançar para o Salsa20 e o ChaCha20, vale a pena situar o RC4 numa classificação mais geral. As cifras de fluxo organizam-se em duas grandes arquitecturas, consoante a forma como produzem o fluxo de chave (Aumasson 2017).

Uma **cifra de fluxo com estado** (*stateful*) inicializa um estado interno a partir da chave e do *nonce* (a fase *Init*), e depois vai **actualizando esse estado** a cada novo bloco do fluxo de chave (a fase *Update*, repetida), de forma sequencial: cada actualização depende da anterior.

O RC4, já apresentado, é exactamente disto: a KSA inicializa o *array* S , e cada iteração da PRGA actualiza S antes de produzir o byte seguinte do fluxo.

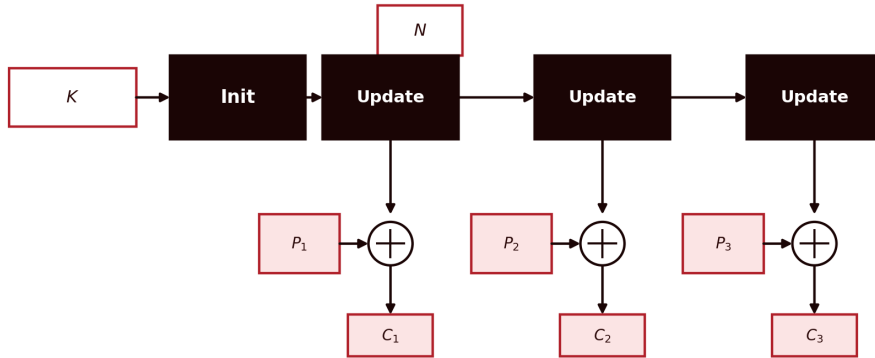


Figura 4.3: Uma cifra de fluxo com estado: a fase Init prepara um estado interno a partir da chave e do nonce; cada fase Update seguinte modifica esse estado e produz um novo troço do fluxo de chave, de forma sequencial.

Uma **cifra de fluxo por contador** (*counter-based*) segue uma lógica diferente: não mantém nenhum estado secreto entre blocos, para além do valor de um contador público. Cada bloco do fluxo de chave calcula-se de novo, directamente a partir da chave, do *nonce* e do valor do contador nesse bloco — sem depender do resultado do bloco anterior.

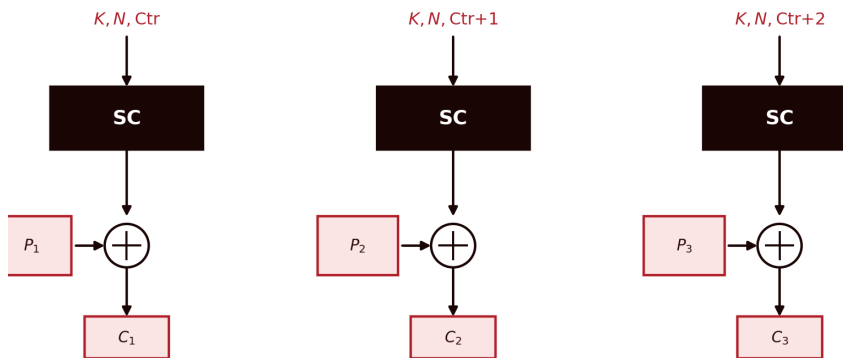


Figura 4.4: Uma cifra de fluxo por contador: cada bloco do fluxo de chave é calculado independentemente, a partir da chave, do nonce e do valor do contador nesse bloco.

Esta diferença arquitectural tem uma consequência prática directa, já referida a propósito do modo CTR no capítulo anterior: como cada bloco de uma cifra por contador é independente dos restantes, pode calcular-se **em paralelo**, ao contrário de uma cifra com estado, em que cada actualização tem de esperar pela anterior. O Salsa20 e o ChaCha20, apresentados de seguida, seguem exactamente esta segunda arquitectura.

4.4 Salsa20

Em resposta às fragilidades do RC4, Daniel J. Bernstein desenhou o **Salsa20** em 2005, submetido ao concurso *eSTREAM* (o projecto europeu ECRYPT para cifras de fluxo), onde foi seleccionado para o portefólio final de algoritmos recomendados para software.

Ao contrário do RC4, o Salsa20 não mantém um estado que evolui byte a byte, nem usa S-boxes ou tabelas de substituição. Baseia-se exclusivamente em três operações, **soma, rotação de bits e XOR** — por isso chamada uma construção **ARX** (*Add-Rotate-XOR*). O seu estado interno é uma matriz 4×4 de palavras de 32 bits (512 bits, 64 bytes no total), organizada em quatro grupos: constantes fixas, a chave (128 ou 256 bits), o *nonce* (64 bits) e um contador (64 bits). Para gerar cada bloco de 64 bytes de fluxo de chave, aplicam-se 20 rondas de uma **função de quarto de ronda**, $QR(a, b, c, d)$:

$$\begin{aligned} b \oplus &= (a + d) \lll 7 & c \oplus &= (b + a) \lll 9 \\ d \oplus &= (c + b) \lll 13 & a \oplus &= (d + c) \lll 18 \end{aligned}$$

e o resultado final soma-se (módulo 2^{32} , palavra a palavra) ao estado inicial.

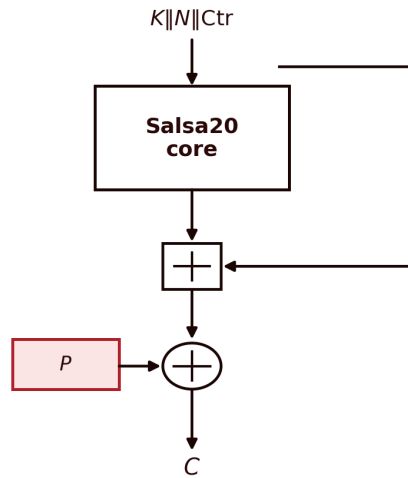


Figura 4.5: O esquema de cifragem do Salsa20 para um bloco de texto claro: o núcleo Salsa20 transforma $K||N||Ctr$, o resultado soma-se (módulo 2^{32} , quadrado com “+”) ao estado inicial para formar o bloco de fluxo de chave, que finalmente se combina por XOR (círculo com “+”) com o texto claro.

Repare-se na distinção entre as duas operações da figura: a soma final ao estado inicial (quadrado) é **adição modular**, não XOR, e é precisamente o que torna a transformação impossível de inverter sem conhecer a chave — se o Salsa20 devolvesse directamente a permutação do núcleo, sem esta soma, seria trivialmente invertível. Só depois desta soma é que o resultado se combina por XOR (círculo) com o texto claro. Como cada bloco de 64 bytes é gerado de forma independente dos restantes (a partir da chave, do *nonce* e do valor do contador nesse bloco), o Salsa20 é inteiramente **paralelizável**, tanto na cifragem como na decifragem.

O melhor ataque conhecido contra o Salsa20 reduz-se a 15 das 20 rondas (Aumasson 2017); a versão completa, de 20 rondas, não tem, até hoje, nenhum ataque prático conhecido, e é considerada segura.

4.5 ChaCha20

O **ChaCha20**, também desenhado por Bernstein, em 2008, é uma variante do Salsa20 com uma função de quarto de ronda modificada, que alcança melhor difusão por ronda: cada palavra de 32 bits participa tanto numa soma como numa rotação em cada um dos quatro passos, o que faz o efeito de cada bit espalhar-se mais depressa pelo resto do estado. É hoje a cifra de fluxo de referência, usada, entre outros, no TLS 1.3, no QUIC, no WireGuard, no SSH e por omissão no OpenSSH desde 2014.

$$\begin{array}{ll} a += b; d \oplus= a; d \lll 16 & c += d; b \oplus= c; b \lll 12 \\ a += b; d \oplus= a; d \lll 8 & c += d; b \oplus= c; b \lll 7 \end{array}$$

As constantes de rotação (16, 12, 8, 7) e a ordem das operações são diferentes das do Salsa20 (7, 9, 13, 18), precisamente para conseguir essa difusão mais rápida. Em relação ao Salsa20, o ChaCha20 normalizado pelo IETF usa ainda um *nonce* maior, de 96 bits (em vez de 64), e um contador mais pequeno, de 32 bits — suficiente para gerar até 256 GB de fluxo de chave a partir de um único *nonce*.

O ChaCha20 é frequentemente combinado com um mecanismo de autenticação de mensagem, o Poly1305, na construção **ChaCha20-Poly1305**, normalizada no RFC 8439 (Nir e Langley 2018): o primeiro bloco gerado pelo ChaCha20 (bloco 0) deriva a chave do Poly1305; os blocos seguintes (1 a N) fornecem o fluxo de chave que cifra a mensagem por XOR; e o Poly1305 calcula, sobre o texto cifrado e sobre eventuais dados associados não cifrados (*additional authenticated data*, AAD, como cabeçalhos de rede), uma marca de autenticação (*tag*) de 128 bits, que acompanha o texto cifrado.

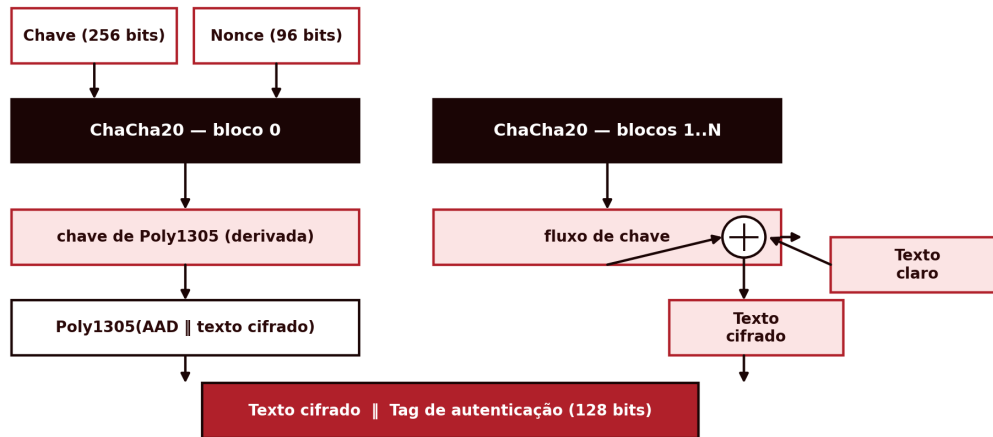


Figura 4.6: A construção ChaCha20-Poly1305: o bloco 0 do ChaCha20 deriva a chave do Poly1305; os blocos seguintes fornecem o fluxo de chave que cifra a mensagem por XOR; o Poly1305 autentica os dados associados (AAD) e o texto cifrado, produzindo uma marca de 128 bits.

Esta é uma **cifra autenticada** (*AEAD*, *Authenticated Encryption with Associated Data*), o mesmo conceito já introduzido a propósito do modo GCM no capítulo anterior: garante simultaneamente confidencialidade e integridade, numa única construção. O ChaCha20, tal como o Salsa20, não tem, até hoje, nenhuma fragilidade estatística conhecida que o torne distinguível de uma fonte aleatória.

4.6 Padding

Uma cifra de fluxo, como se viu, dispensa qualquer noção de bloco: processa a mensagem byte a byte, seja qual for o seu comprimento. As cifras de bloco em modos como o CBC ou o ECB não têm essa liberdade: exigem que a mensagem seja um múltiplo exacto do tamanho do bloco antes de a cifrar. Quando o último bloco de uma mensagem não está completo, o algoritmo de cifragem não tem bytes suficientes para o processar.

O **padding** resolve isto acrescentando bytes extra ao fim da mensagem, antes da cifragem, até completar o bloco. O esquema mais comum é o **PKCS#7**: se faltam n bytes para completar o último bloco, preenchem-se esses n bytes com o valor n (em hexadecimal, $0xN$). Por exemplo, se faltam 10 bytes para completar o bloco, acrescentam-se dez bytes, todos com o valor $0xA$. Do lado da decifragem, basta ler o valor do último byte recebido para saber exactamente quantos bytes de *padding* remover, sem qualquer ambiguidade.

Vale a pena recordar, como já ficou registado na tabela de modos de operação do capítulo anterior, que o **CBC** é precisamente um dos modos que depende deste mecanismo — e que uma validação de *padding* mal implementada, que revele a um atacante, através de uma mensagem de erro ou de uma diferença de tempo de resposta, se o *padding* recebido era ou não válido, pode ser explorada para decifrar mensagens inteiras sem nunca conhecer a chave, byte a byte, através do chamado **ataque de oráculo de padding** (*padding oracle attack*). A lição de fundo é a mesma que já apareceu antes neste livro: um mecanismo matematicamente correcto pode ainda assim falhar por causa de como é implementado.

4.7 Síntese do Capítulo

Este capítulo apresentou a segunda grande família de cifras simétricas:

1. Uma **cifra de fluxo** gera um fluxo de chave pseudo-aleatório, combinado com o texto claro por XOR, byte a byte, sem noção de bloco — o equivalente pseudo-aleatório do One-Time Pad
2. **Nunca reutilizar o mesmo par (chave, nonce)**: fazê-lo permite o ataque do *two-time pad*, que cancela o fluxo de chave e expõe o XOR dos dois textos claros
3. O **RC4**, sem *nonce* incorporado e com um fluxo de chave estatisticamente enviesado nos primeiros bytes, acumulou fragilidades (Fluhrer–Mantin–Shamir, 2001; AlFardan et al., 2013) que o tornaram inseguro, e está hoje banido de normas como o TLS
4. O **Salsa20** e o **ChaCha20**, construções ARX (soma, rotação, XOR) desenhadas por Bernstein, não têm fragilidades práticas conhecidas; o ChaCha20, com melhor difusão por ronda, é hoje a cifra de fluxo de referência, frequentemente usada em conjunto com o Poly1305 numa construção AEAD
5. O **padding** (nomeadamente o esquema PKCS#7) permite às cifras de bloco em modos como o CBC processar mensagens de qualquer comprimento, mas a sua implementação incorrecta pode originar ataques de oráculo de *padding*

4.8 Para Pensar

O RC4 não tem nenhum mecanismo de *nonce* incorporado; o ChaCha20 exige um *nonce* de 96 bits para cada mensagem. Tendo em conta o ataque do *two-time pad* apresentado neste capítulo, que consequência prática tem esta diferença de desenho para quem implementa um protocolo de comunicação com qualquer uma destas duas cifras?

Referências

- AlFardan, Nadhem J., Daniel J. Bernstein, Kenneth G. Paterson, Bertram Poettering, e Jacob C. N. Schuldt. 2013. «On the Security of RC4 in TLS». Em *22nd USENIX Security Symposium*.
- Aumasson, Jean-Philippe. 2017. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press.
- Böck, Hanno, Aaron Zauner, Sean Devlin, Juraj Somorovsky, e Philipp Jovanovic. 2016. «Nonce-Disrespecting Adversaries: Practical Forgery Attacks on GCM in TLS». Em *USENIX Workshop on Offensive Technologies (WOOT)*.
- Du, Wenliang. 2022. *Computer Security: A Hands-on Approach*. 3.^a ed. CreateSpace / Self-published.
- Dworkin, Morris. 2001. «Recommendation for Block Cipher Modes of Operation: Methods and Techniques». SP 800-38A. NIST.
- . 2007. «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC». SP 800-38D. NIST.
- Fluhrer, Scott, Itsik Mantin, e Adi Shamir. 2001. «Weaknesses in the Key Scheduling Algorithm of RC4». Em *Selected Areas in Cryptography (SAC 2001)*.
- Kerckhoffs, Auguste. 1883. «La cryptographie militaire». *Journal des Sciences Militaires* 9: 5–38.
- Martin, Keith. 2017. *Everyday Cryptography*. 2.^a ed. Oxford University Press.
- National Institute of Standards and Technology. 2001. «Advanced Encryption Standard (AES)». FIPS PUB 197. NIST.
- Nir, Yoav, e Adam Langley. 2018. «ChaCha20 and Poly1305 for IETF Protocols». RFC 8439. IETF.
- Popov, Andrei. 2015. «Prohibiting RC4 Cipher Suites». RFC 7465. IETF.
- Shannon, Claude E. 1949. «Communication Theory of Secrecy Systems». *Bell System Technical Journal* 28 (4): 656–715.
- Stallings, William. 2020. *Cryptography and Network Security: Principles and Practice*. 8.^a ed. Pearson.

5

Hashing — Propriedades, Ataques, SHA e HMAC

Conteúdo por desenvolver.

6

Criptografia Assimétrica — Fundamentos e RSA

Conteúdo por desenvolver.

7

Diffie-Hellman e Assinaturas Digitais

Conteúdo por desenvolver.

8

Criptografia de Curva Elíptica (ECC)

Conteúdo por desenvolver.

9

Infraestrutura de Chave Pública (PKI)

Conteúdo por desenvolver.

10

TLS e PGP — Esquemas Criptográficos Aplicados

Conteúdo por desenvolver.